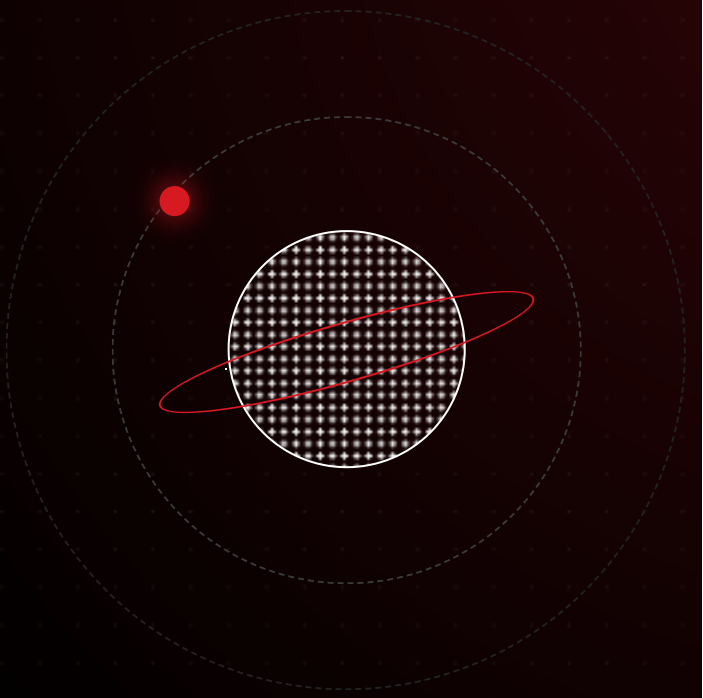
ceti
CENTRO DE ENSEÑANZA TÉCNICA INDUSTRIAL



ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE · IEEE 830

SRS

ERP Café Petrova Line

FINANZAS · COMPRAS Y PROVEEDORES · VENTAS · RRHH · INVENTARIO
SEGURIDAD CON MIDDLEWARE · PAYPAL · FACTURA XML POR CORREO

ALUMNO
Francisco Yael Pérez Palacios

REGISTRO
23300664

GRUPO
6A1

FECHA
28/10/2026

PROFESORA
Paola Fernanda Ponce Villalvazo

INSTITUCIÓN
Centro de Enseñanza Técnica Industrial

00 Contenido

01 Introducción
02 Descripción general
03 Requisitos funcionales
04 Interfaces externas
05 Requisitos no funcionales
06 Modelado del sistema
07 Implementación de la seguridad
08 Evidencia de funcionamiento
09 Repositorio y ejecución
10 Matriz de trazabilidad

01 Introducción

• 1.1 PROPÓSITO

Este documento especifica los requisitos del **ERP web de Café Petrova Line**, una cafetería con temática espacial (inspirada en *Proyecto Hail Mary*): el menú es ”combustible”, las ventas son la ”Bitácora de Misión”, las compras son ”Manifiestos de Carga” y el personal es la ”Tripulación”. Sirve de guía para el desarrollo, las pruebas y la revisión en clase.

• 1.2 ALCANCE

El sistema cubre los cinco módulos que pide la práctica, más dos de apoyo:

MÓDULO	NOMBRE EN EL SISTEMA	QUÉ RESUELVE
Finanzas y contabilidad	Balance de Energía	Movimientos, utilidad por fechas, ventas por día, formas de pago, más vendidos
Compras y Proveedores	Manifiestos de Carga	Proveedores, órdenes de compra, recepción al inventario y pago
Ventas	Bitácora de Misión	Punto de venta, cobro con PayPal, factura XML por correo, corte de caja
RRHH	Tripulación	Expediente de empleados, rotaciones de turno y nómina estimada
Inventario	Bodega de Combustible	Catálogo, entradas, merma y alertas de stock mínimo
Apoyo	Programa Amaze / Usuarios	Clientes con correo y puntos; cuentas y roles de acceso

El sistema lo usa el personal de la cafetería desde el navegador. No es una tienda en línea para clientes.

1.3 DEFINICIONES

TÉRMINO	SIGNIFICADO
ERP	Sistema de planificación de recursos empresariales
Middleware	Función de Express que se ejecuta antes de la ruta y puede dejar pasar o cortar la petición
JWT	Token firmado con los datos de la sesión (id, rol); se guarda en una cookie httpOnly
Ticket / Folio PL-n	Una venta con uno o varios productos
Manifiesto MC-n	Una orden de compra a un proveedor
Energía neta	Ingresos menos egresos (utilidad) del periodo
Factura XML	Comprobante interno con estructura tipo CFDI: emisor, receptor, conceptos, IVA y totales
Sandbox	Ambiente de pruebas de PayPal: usa dinero ficticio
Proxy	Configuración de Angular que reenvía /api al puerto 3200 del backend

1.4 REFERENCIAS

- IEEE Std 830-1998, prácticas recomendadas para especificación de requisitos de software.
- Documentación de Angular 22, Express 4, mysql2, jsonwebtoken, bcryptjs y Nodemailer.
- PayPal Developer: Orders API v2 y JavaScript SDK (modo sandbox).
- Estructura de referencia: ERP web de cafetería de Carlos Gael Barbosa Mendoza (Café Sanrio), adaptada a la temática Petrova.

02 Descripción general

• 2.1 PERSPECTIVA DEL PRODUCTO

Es una aplicación de una sola página en **Angular** que consume una **API REST propia** en **Node.js + Express** conectada a **MySQL**. El backend es el único que conoce las claves de PayPal y del correo; el navegador solo habla con `/api`.

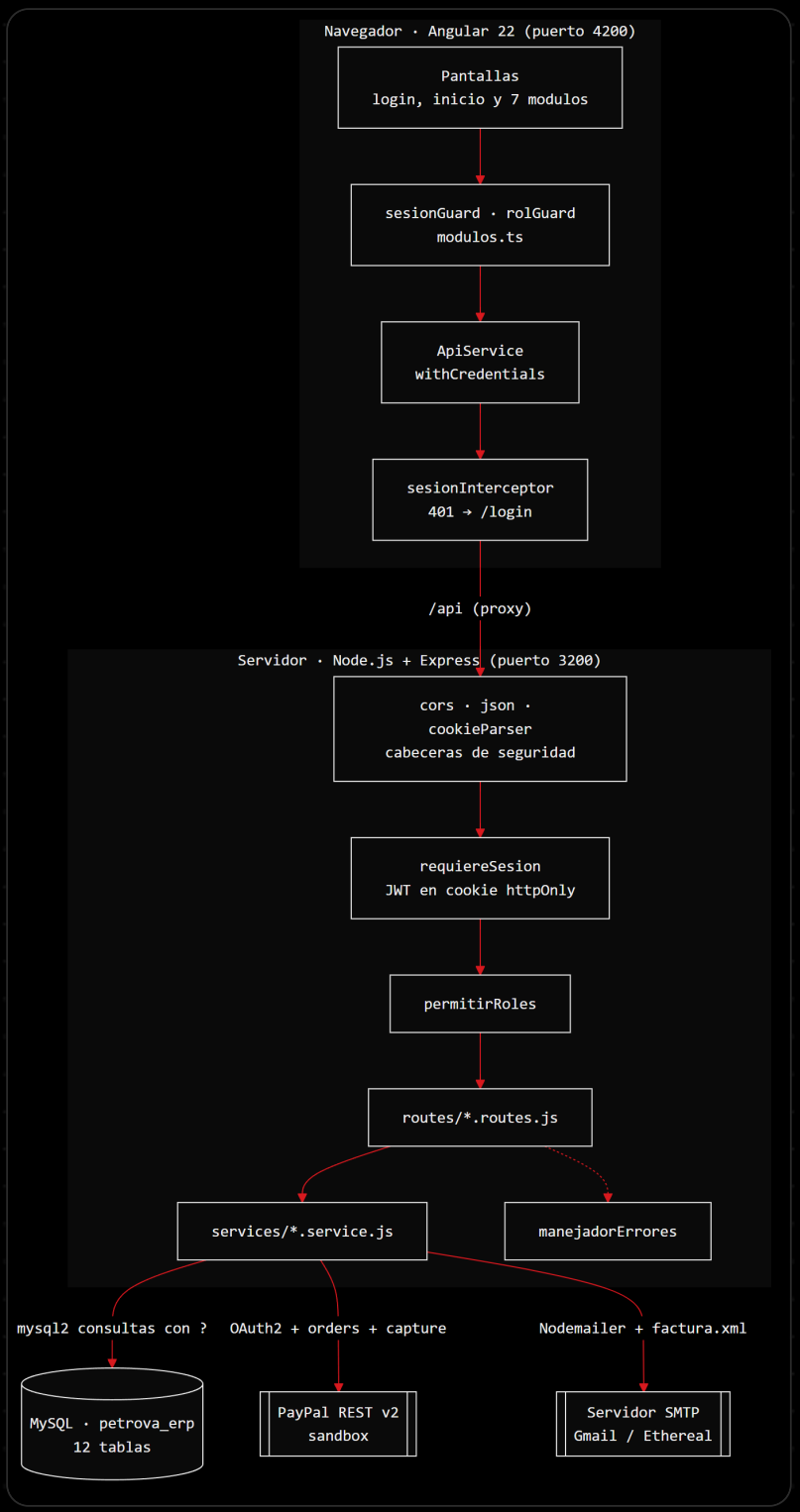


Figura 1. Arquitectura: front, cadena de middleware, servicios y servicios externos

2.2 USUARIOS Y ROLES

ROL	QUIÉN ES	MÓDULOS QUE ABRE
`comandante`	Dueño o encargado (R. Grace)	Todos
`cajero`	Caja y barra (Ilyukhina Petrova)	Ventas, Clientes
`almacenista`	Bodega (Yao Li-Jie)	Inventario, Compras
`tripulacion`	Encargado de personal (Dimitri Voloshin)	Tripulación

Los permisos se escriben una sola vez en `frontend/src/app/modulos.ts` (de ahí salen el menú, las tarjetas del inicio y el `rolGuard`) y el backend los vuelve a revisar en cada petición con `permitirRoles` en `server.js`.

2.3 RESTRICCIONES

- Frontend en Angular 22 con componentes standalone, signals, guards funcionales e interceptor HTTP.
- Backend en Express con la misma estructura de los apuntes: `server.js`, `config`, `models`, `routes` y `services`, más `middleware`.
- Base de datos MySQL (XAMPP); un solo script `sql/petrova_erp.sql` crea tablas y datos de prueba.
- Claves y contraseñas en `.env`, que no se sube al repositorio.
- PayPal en modo sandbox; moneda MXN.

2.4 SUPUESTOS Y DEPENDENCIAS

- Los precios del menú no incluyen IVA; el ticket y la factura agregan el 16 %.
- La factura XML es un comprobante interno con estructura tipo CFDI; no está timbrada ante el SAT (RFC del receptor genérico XAXX010101000).
- Hay internet para PayPal y para el correo. Sin credenciales SMTP el backend usa una cuenta de prueba de Ethereal.

03 Requisitos funcionales

Cada requisito tiene un identificador para rastrearlo en las pruebas y en la matriz de trazabilidad (sección 10).

• 3.1 ACCESO Y SEGURIDAD

ID	REQUISITO	ROL	DÓNDE
RF-01	Iniciar sesión con correo y contraseña; los errores se muestran sin decir cuál dato falló	Todos	POST /usuarios/login
RF-02	Mantener la sesión al recargar la página (8 h) y cerrarla con el botón Salir	Todos	GET /usuarios/sesion, POST /logout
RF-03	Mostrar en el menú y en el inicio solo los módulos que el rol puede abrir; bloquear por URL los demás	Todos	modulos.ts, rolGuard
RF-04	Dar de alta usuarios ligados a un tripulante con rol; activarlos, desactivarlos o borrarlos (no a sí mismo)	Comandante	/usuarios

• 3.2 VENTAS (BITÁCORA DE MISIÓN)

ID	REQUISITO	ROL	DÓNDE
RF-05	Armar un ticket con varios productos y cantidades sin pasar la existencia disponible	Comandante, cajero	/ventas
RF-06	Asignar la venta a un socio Amaze o a público en general	Comandante, cajero	/ventas
RF-07	Calcular subtotal, IVA 16 %, total y cambio en efectivo	Comandante, cajero	Venta.calcularTotal()
RF-08	Cobrar en efectivo, tarjeta o transferencia	Comandante, cajero	POST /ventas
RF-09	Cobrar con PayPal (crear orden, aprobar y capturar); la venta se guarda solo si el pago se completó	Comandante, cajero	/paypal/orden, /paypal/captura
RF-10	Al vender: guardar detalle, descontar existencia, registrar el ingreso en Finanzas y sumar 1 punto por cada $10	Sistema	venta.service
RF-11	Consultar ventas registradas y el corte de caja del día del usuario conectado	Comandante, cajero	GET /ventas, /ventas/corte
RF-12	Descargar la factura de una venta en formato XML	Comandante, cajero	GET /ventas/:id/xml

RF-13	Enviar la factura XML como adjunto al correo del cliente	Comandante, cajero	POST /ventas/:id/enviar
RF-14	Cancelar una venta regresando la existencia y quitando el ingreso	Comandante	DELETE /ventas/:id
RF-15	Registrar, editar y buscar socios Amaze con teléfono, correo, puntos y nivel (Piano, Mezzo, Forte)	Comandante, cajero	/clientes

• 3.3 INVENTARIO (BODEGA DE COMBUSTIBLE)

ID	REQUISITO	ROL	DÓNDE
RF-16	Dar de alta, editar y borrar productos con código, temperatura, kcal, precio y stock mínimo	Comandante, almacenista	/productos
RF-17	Registrar entradas de mercancía que suben la existencia	Comandante, almacenista	POST /productos/:id/entrada
RF-18	Registrar merma con motivo (caducidad, daño, derrame, error) sin pasar la existencia	Comandante, almacenista	POST /productos/merma
RF-19	Marcar en alerta los productos en su stock mínimo y mostrar el valor en bodega	Comandante, almacenista	/inventario

• 3.4 COMPRAS Y PROVEEDORES (MANIFIESTOS DE CARGA)

ID	REQUISITO	ROL	DÓNDE
RF-20	Registrar proveedores con teléfono y correo	Comandante, almacenista	/compras/proveedores
RF-21	Generar un manifiesto con varios productos, cantidad y costo unitario	Comandante, almacenista	POST /compras
RF-22	Recibir el manifiesto una sola vez sumando los productos al inventario	Comandante, almacenista	PUT /compras/:id/recibir
RF-23	Marcar el manifiesto como pagado registrando el egreso en Finanzas	Comandante, almacenista	PUT /compras/:id/pago
RF-24	Mostrar el total por pagar a proveedores y los manifiestos en tránsito	Comandante, almacenista	/compras

• 3.5 FINANZAS Y CONTABILIDAD (BALANCE DE ENERGÍA)

ID	REQUISITO	ROL	DÓNDE

RF-25	Registrar ingresos y egresos manuales con categoría, concepto, monto y fecha	Comandante	POST /finanzas/movimientos
RF-26	Calcular ingresos, egresos y energía neta (utilidad) en un rango de fechas	Comandante	GET /finanzas/balance
RF-27	Graficar las ventas de los últimos 7 días	Comandante	GET /finanzas/ventas-por-dia
RF-28	Reportar ventas por forma de pago y los 10 productos más vendidos	Comandante	/formas-pago, /mas-vendidos

• 3.6 RECURSOS HUMANOS (TRIPULACIÓN)

ID	REQUISITO	ROL	DÓNDE
RF-29	Dar de alta, editar y dar de baja tripulantes con puesto, teléfono, ingreso y sueldo por hora	Comandante, tripulación	/empleados
RF-30	Asignar y quitar rotaciones por día y horario; calcular la nómina semanal estimada	Comandante, tripulación	/empleados/rotaciones

04 Interfaces externas

• 4.1 PAYPAL (SERVICIO EXTERNO)

El backend pide un token a `https://api-m.sandbox.paypal.com/v1/oauth2/token` con `PAYPAL_CLIENT_ID` y `PAYPAL_CLIENT_SECRET`, crea la orden en `/v2/checkout/orders` con el total que **calcula él mismo** desde la base y, cuando el cliente aprueba en la ventana de PayPal, la captura con `/v2/checkout/orders/{id}/capture`. Solo si la captura regresa `COMPLETED` se guarda la venta con `forma_pago = 'paypal'` y el id de la captura. El front solo recibe el *client id* (público) para pintar los botones con el SDK de JavaScript.

• 4.2 CORREO CON LA FACTURA XML (SERVICIO EXTERNO)

Con **Nodemailer** y un servidor SMTP (Gmail con contraseña de aplicación: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`) se manda al correo del cliente un mensaje con el archivo `factura-PL-n.xml` adjunto. Si el `.env` no trae SMTP, se crea una cuenta de prueba en **Ethereal** y la API responde con la liga para ver el correo. Factura real generada por el sistema para la venta de la evidencia:

FACTURA-PL-7.XML — GET /API/VENTAS/7/XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Factura folio="PL-7" fecha="2026-10-28" moneda="MXN" formaPago="efectivo">
  <Emisor nombre="Cafe Petrova Line" rfc="CPL260101AB1" regimen="601" lugarExpedicion="44100"/>
  <Receptor nombre="Rocky Grace" rfc="XAXX010101000" correo="rocky@eridani.mx"/>
  <Atendio nombre="R. Grace"/>
  <Conceptos>
    <Concepto clave="P-001" descripcion="Astrophage Latte" cantidad="2" valorUnitario="62.00" importe="124.00"/>
    <Concepto clave="P-008" descripcion="Muffin Hail Mary" cantidad="1" valorUnitario="42.00" importe="42.00"/>
  </Conceptos>
  <Impuestos tipo="IVA" tasa="0.16" importe="26.56"/>
  <Totales subtotal="166.00" iva="26.56" total="192.56"/>
</Factura>
```

• 4.3 API REST PROPIA

Todas las rutas empiezan con `/api` y responden JSON. Los errores siempre traen `{ "mensaje": "..." }`.

RUTA BASE	ACCESO	OPERACIONES
/usuarios	login y logout públicos; resto comandante	POST /login · GET /sesion · POST /logout · GET, POST · PUT /:id/estado · DELETE /:id
/productos	sesión (leer) · comandante y almacenista (escribir)	GET · GET /alertas · GET /:id · POST · PUT /:id · DELETE /:id · POST /:id/entrada · POST /merma
/ventas	comandante, cajero	GET · POST · GET /corte · GET /:id · GET /:id/xml · POST /:id/enviar · DELETE /:id
/paypal	comandante, cajero	GET /config · POST /orden · POST /captura
/clientes	comandante, cajero	GET · POST · PUT /:id · PUT /:id/puntos
/compras	comandante, almacenista	GET, POST, PUT, DELETE /proveedores · GET · POST · GET /:id · PUT /:id/recibir · PUT /:id/pago
/finanzas	comandante	GET, POST /movimientos · GET /balance?desde&hasta · /mas-vendidos · /ventas-por-dia · /formas-pago
/empleados	comandante, tripulación	GET · POST · PUT /:id · DELETE /:id · GET /rotaciones/lista · POST /rotaciones · DELETE /rotaciones/:id

• 4.4 INTERFAZ DE USUARIO

Diseño estilo **Nothing** (negro, matriz de puntos, tipografía Doto, acentos rojos #D71921) llevado al espacio: campo de estrellas, planeta de puntos con anillo, órbitas punteadas y códigos de módulo (VNT, INV, CMP, FIN, RRHH). Las tablas se desplazan en celular y los formularios pasan a una columna.

05 Requisitos no funcionales

• 5.1 SEGURIDAD (MIDDLEWARE)

ID	REQUISITO	CÓMO SE CUMPLE
RNF-01	Solo usuarios con sesión usan la API	`requiereSesion` valida el JWT de la cookie; sin token o token alterado → 401
RNF-02	Cada rol entra solo a sus módulos	`permitirRoles(...)` en `server.js` → 403; en Angular `rolGuard` y menú desde `modulos.ts`
RNF-03	La sesión no se puede leer desde JavaScript	Cookie `httpOnly`, `sameSite: lax`, `secure` en producción, vence a las 8 h
RNF-04	Contraseñas nunca en texto	Hash con bcrypt (10 rondas); el login compara con `bcrypt.compare`
RNF-05	Freno a ataques de fuerza bruta	`limitarIntentos`: 5 logins por minuto por IP → 429
RNF-06	Sin inyección SQL	Todas las consultas usan parámetros `?` de mysql2
RNF-07	Precios que no se alteran desde el navegador	`calcularTotales()` toma precio y existencia de la base antes de cobrar o crear la orden de PayPal
RNF-08	Claves fuera del código	`.env` (excluido en `.gitignore`); el secret de PayPal y la contraseña SMTP nunca llegan al front
RNF-09	Solo el front autorizado llama a la API	`cors` con origen `http://localhost:4200` y `credentials: true`
RNF-10	Cabeceras de seguridad	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`
RNF-11	Errores claros sin tumbar el servidor	`manejadorErrores`: JSON inválido 400, duplicado 409, dependencia 409; `rutaNoEncontrada` 404
RNF-12	Sesión vencida en el front	`sesionInterceptor` manda al login cuando la API responde 401

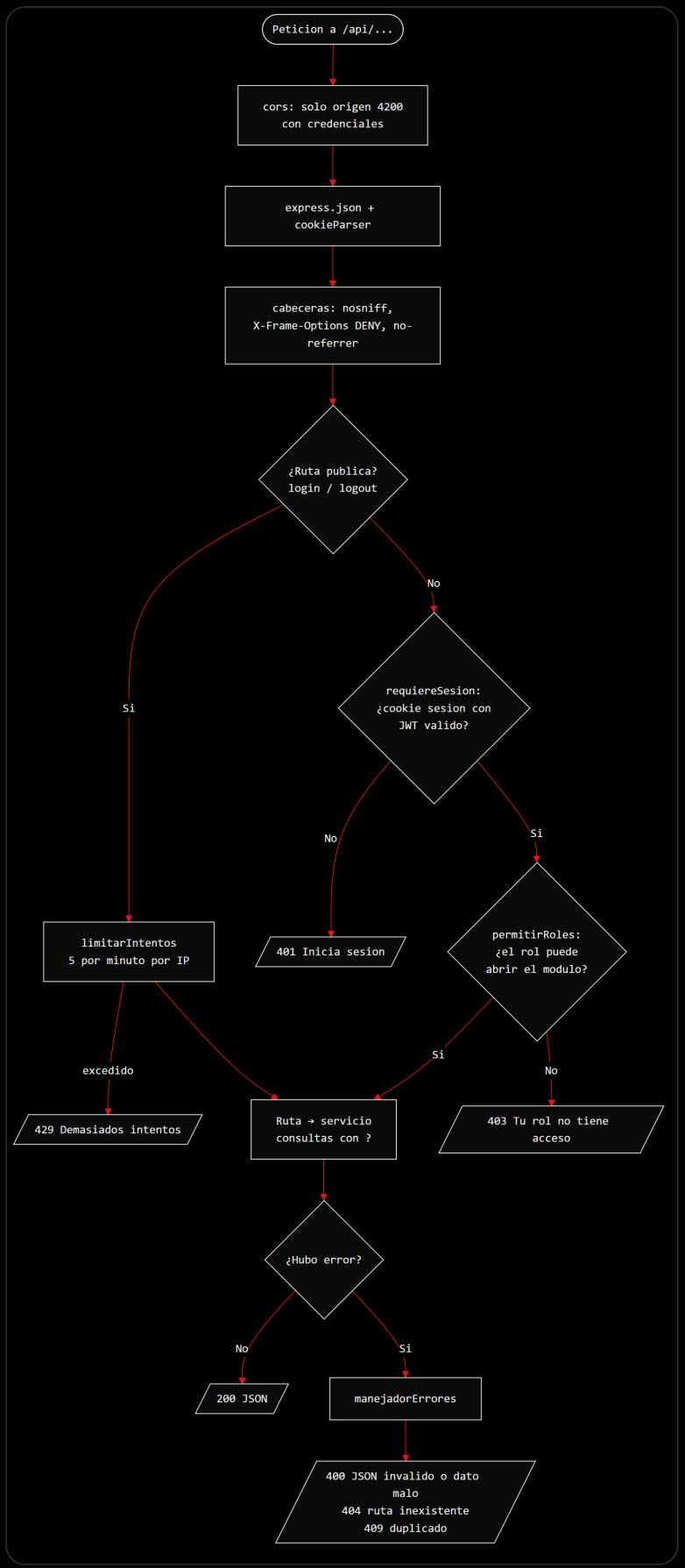


Figura 2. Recorrido de una petición por la cadena de middleware

5.2 USABILIDAD

- Interfaz en español, con la temática de la nave en cada módulo.
- Cada acción muestra un mensaje de éxito (verde) o de error (rojo) con el texto que manda el backend.
- Responsiva: probada a 1366 px (laptop) y 390 px (celular).

5.3 RENDIMIENTO Y DISPONIBILIDAD

- Las pantallas cargan sus datos en menos de 2 segundos con la base local.
- Un error en una consulta no detiene el servidor: lo atrapa el manejador de errores.

5.4 MANTENIBILIDAD

- Se conservó el código de la práctica anterior: mismas rutas y servicios con callbacks, mismas clases en `modelos.ts` y mismas pantallas; se agregó encima la seguridad, la API real y los servicios externos.
- Un solo `ApiService` en Angular y un solo archivo de permisos (`modulos.ts`).

06 Modelado del sistema

• 6.1 CASOS DE USO

Cajero, almacenista, encargado de tripulación y comandante son especializaciones de **Usuario** (todos inician y cierran sesión). El comandante además puede ejecutar los casos de todos los demás roles.

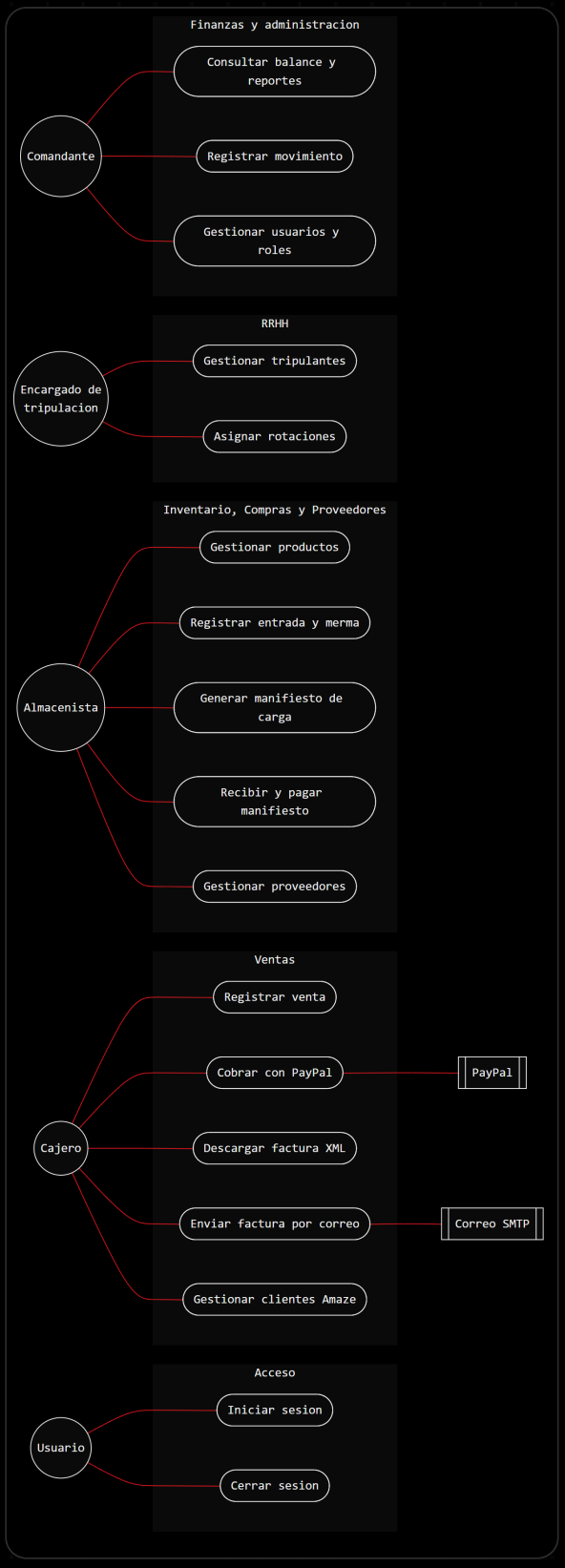


Figura 3. Diagrama de casos de uso por módulo

• 6.2 DIAGRAMA DE CLASES (FRONTEND/SRC/APP/MODELOS.TS)

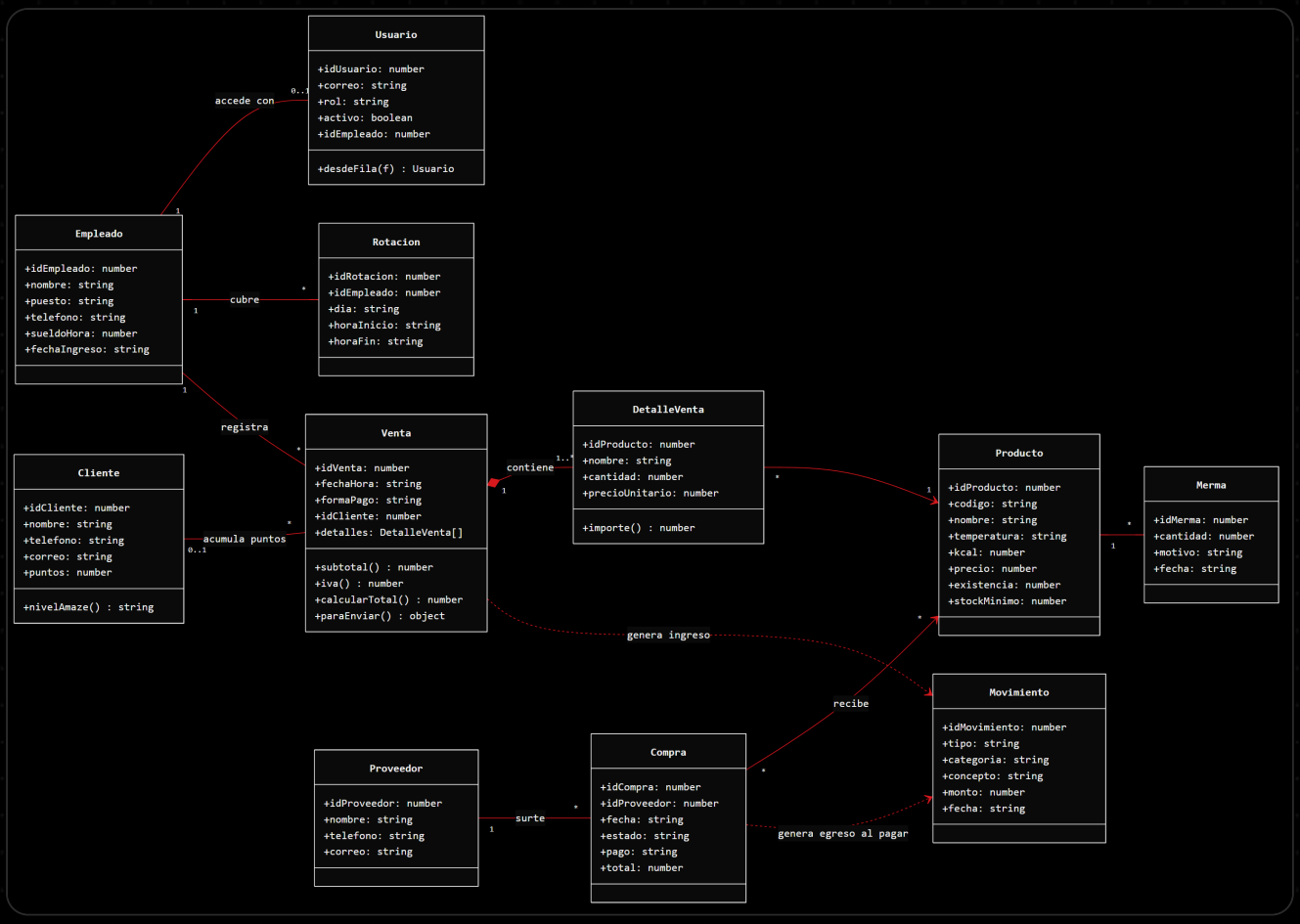


Figura 4. Clases del dominio y sus relaciones

• 6.3 MODELO ENTIDAD-RELACIÓN (PETROVA_ERP)

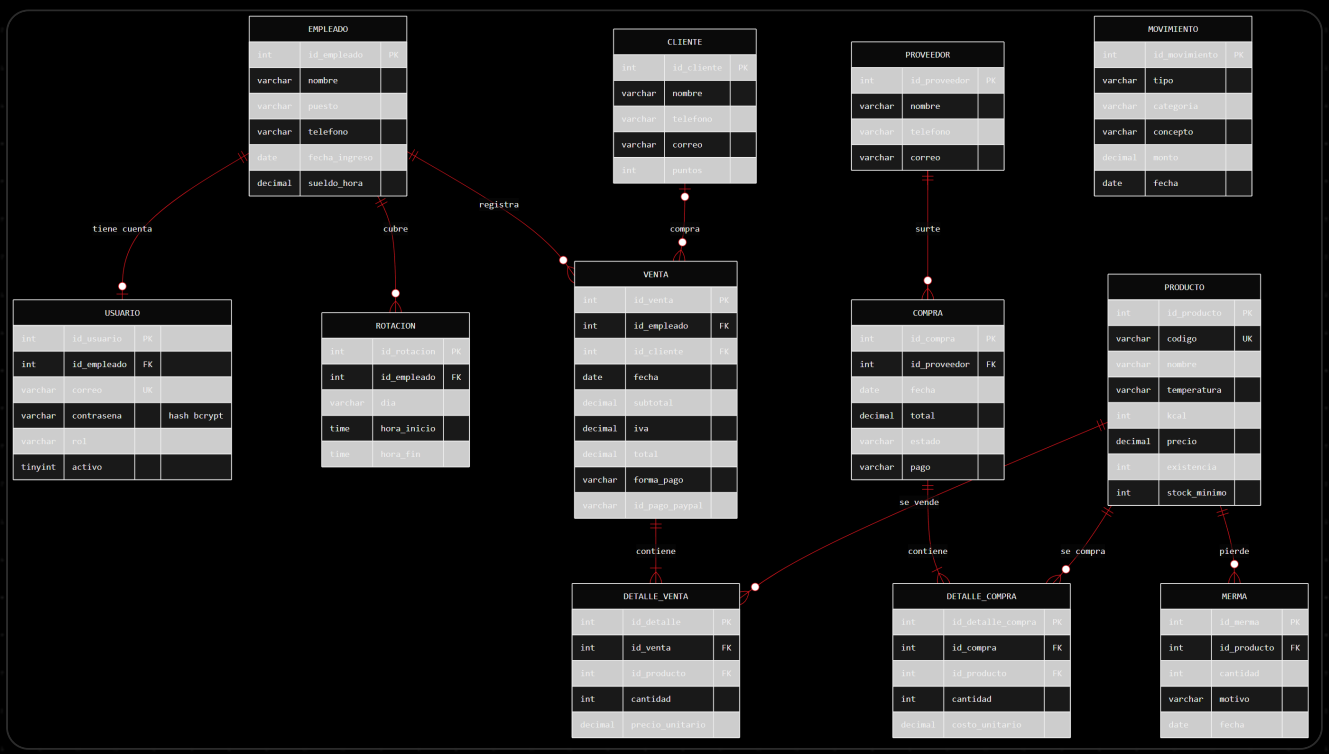


Figura 5. Las 12 tablas de la base de datos

• 6.4 SECUENCIA: INICIO DE SESIÓN Y CONTROL DE ACCESO

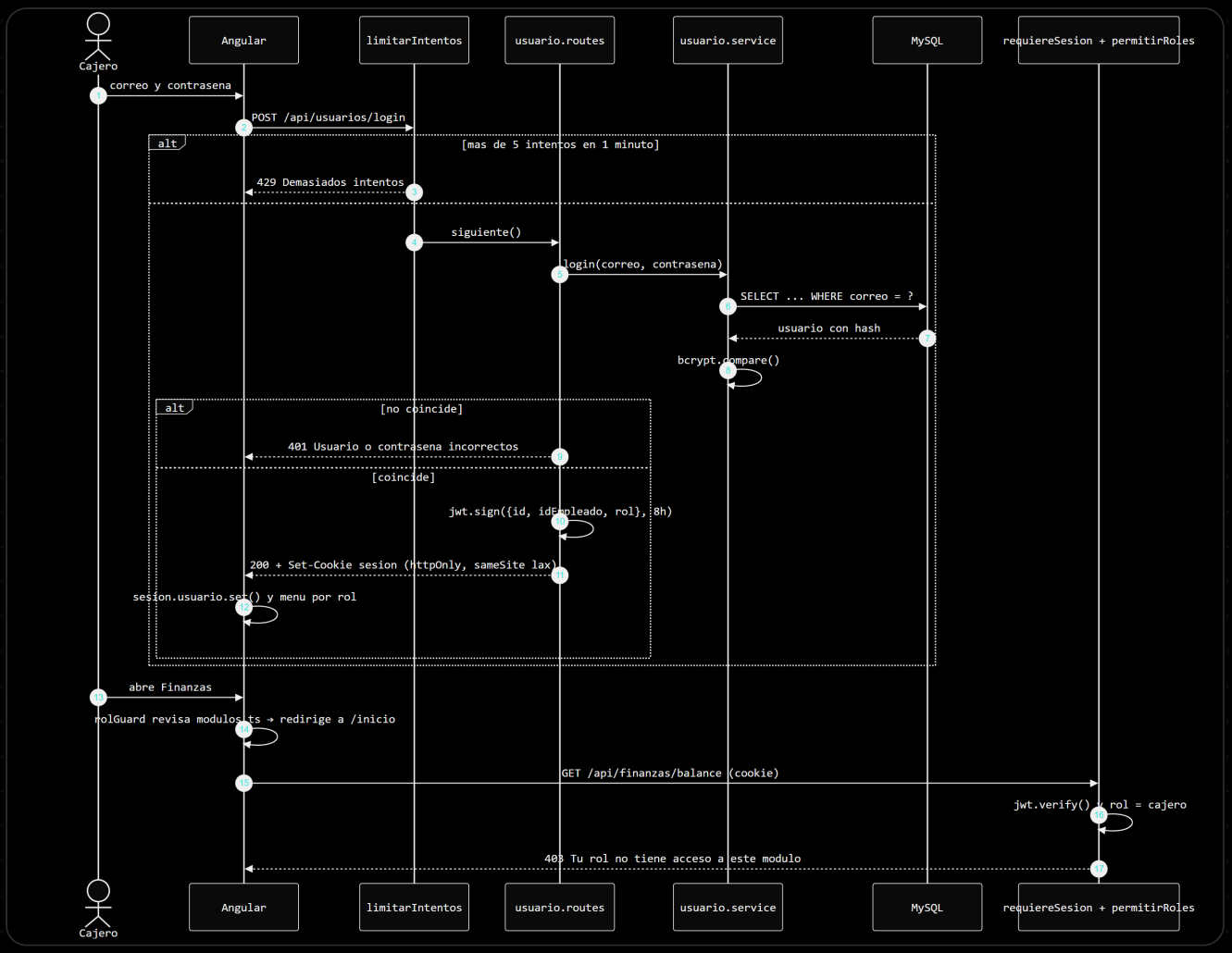


Figura 6. Login con bcrypt y JWT, y bloqueo por rol (403)

• 6.5 SECUENCIA: COBRO CON PAYPAL

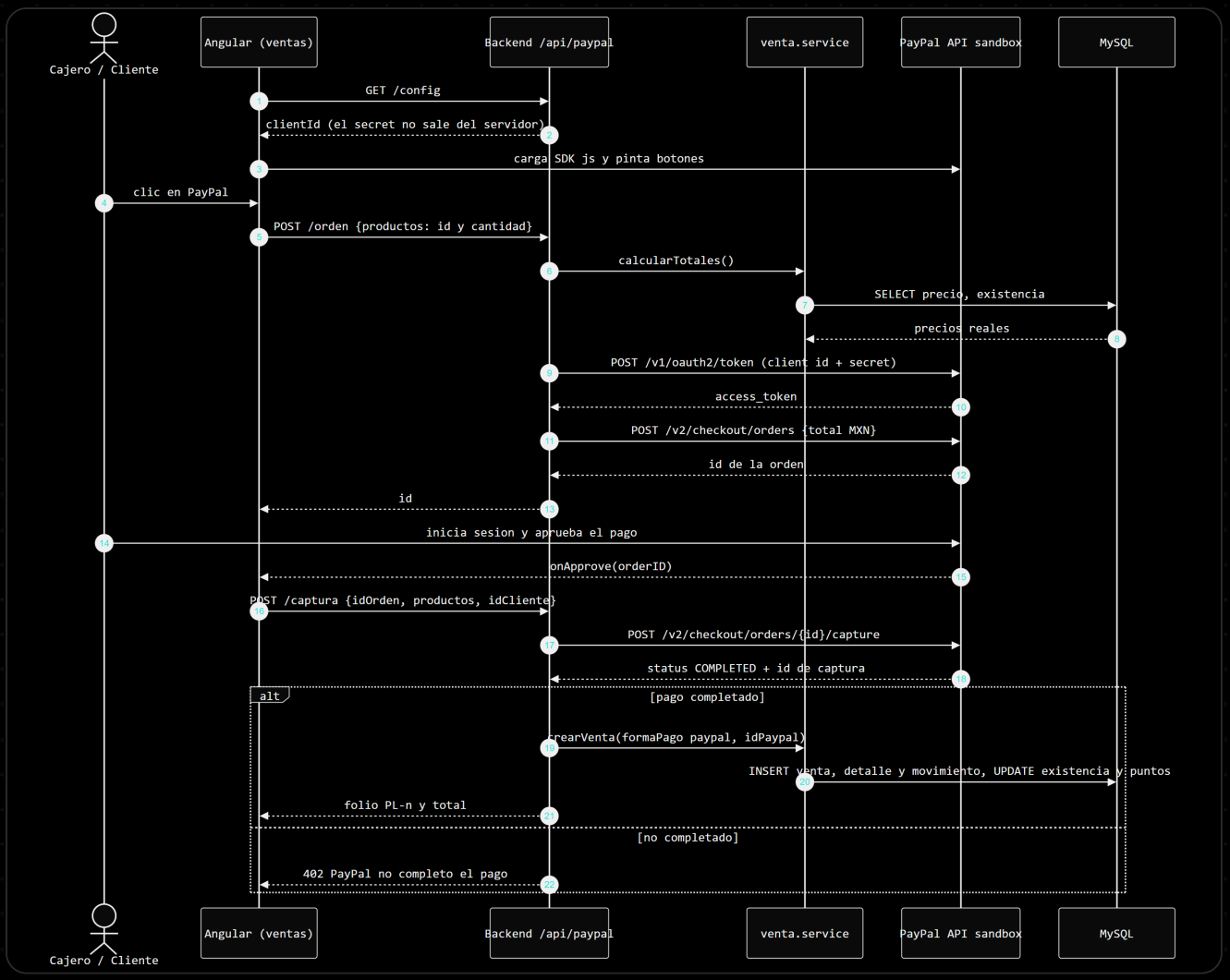


Figura 7. Orden, aprobación y captura en PayPal sandbox

• 6.6 SECUENCIA: FACTURA XML POR CORREO

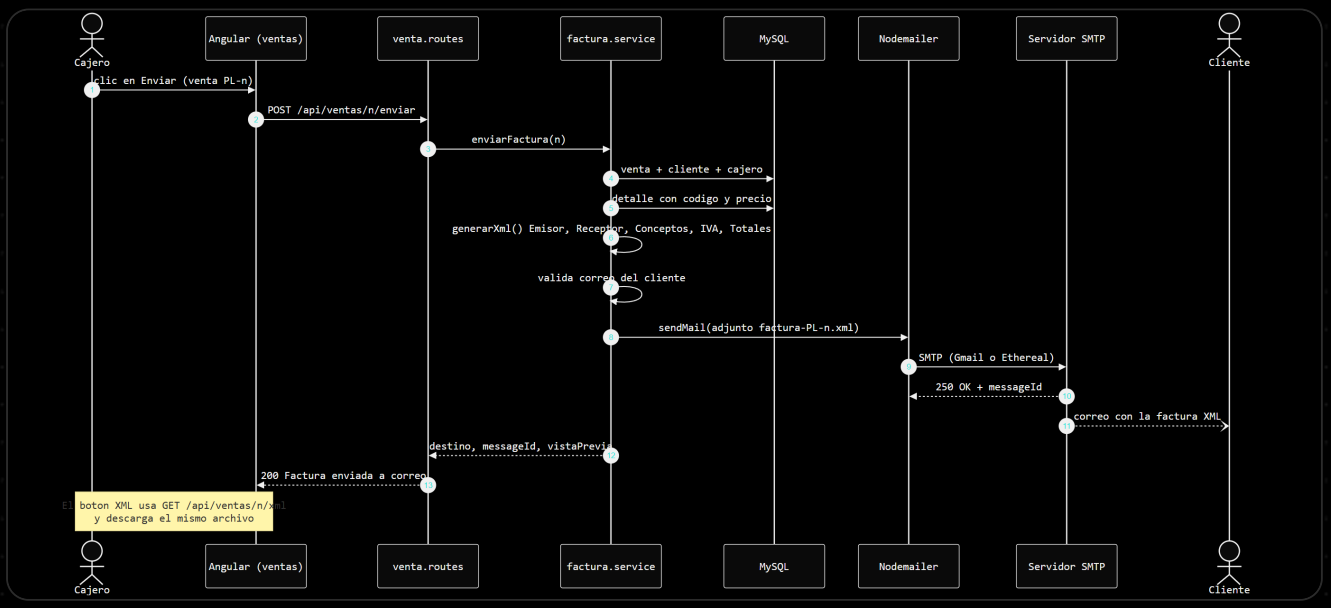


Figura 8. Generación del XML y envío con Nodemailer

● 6.7 ACTIVIDAD: REGISTRAR UNA VENTA

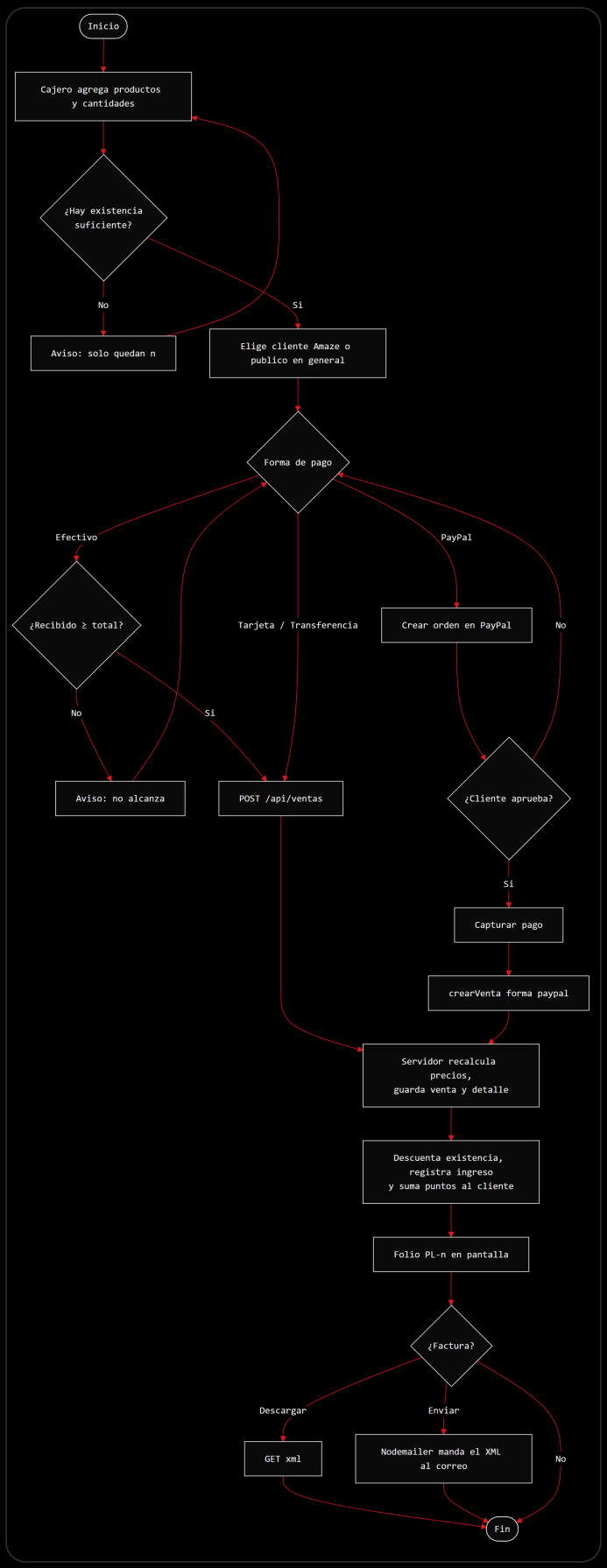


Figura 9. Flujo completo de una venta

• 6.8 MAPA DE NAVEGACIÓN

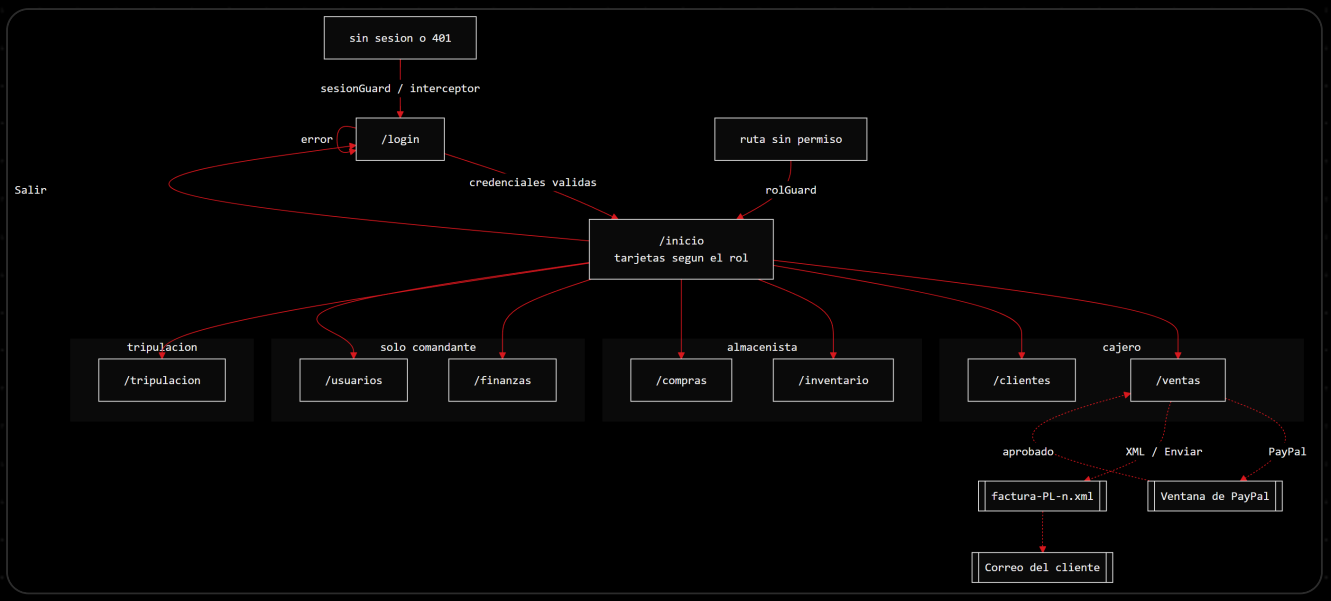


Figura 10. Pantallas, roles y redirecciones de los guards

07 Implementación de la seguridad

Estos son los archivos donde vive la seguridad del backend. `server.js` monta cada módulo con su cadena `requiereSesion → permitirRoles`; los middleware están en la carpeta `backend/middleware`.

• BACKEND/SERVER.JS

BACKEND/SERVER.JS — LÍNEAS 1–46 DE 50 · CONTINÚA →

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const { requiereSesion, permitirRoles } = require('./middleware/auth.middleware');
const { rutaNoEncontrada, manejadorErrores } = require('./middleware/errores.middleware');

const app = express();

// Solo el front de Angular puede llamar a la API y mandar la cookie
app.use(cors({ origin: process.env.ORIGEN_FRONT || 'http://localhost:4200', credentials: true }));
app.use(express.json());
app.use(cookieParser());

// Cabeceras basicas de seguridad
app.use((peticion, respuesta, siguiente) => {
  respuesta.set('X-Content-Type-Options', 'nosniff');
  respuesta.set('X-Frame-Options', 'DENY');
  respuesta.set('Referrer-Policy', 'no-referrer');
  siguiente();
});

const usuarioRoutes = require('./routes/usuario.routes');
const productoRoutes = require('./routes/producto.routes');
const ventaRoutes = require('./routes/venta.routes');
const compraRoutes = require('./routes/compra.routes');
const finanzasRoutes = require('./routes/finanzas.routes');
const empleadoRoutes = require('./routes/empleado.routes');
const clienteRoutes = require('./routes/cliente.routes');
const paypalRoutes = require('./routes/paypal.routes');

// usuarios: el login es publico, lo demas lo protege la propia ruta
app.use('/api/usuarios', usuarioRoutes);

// Cada modulo pide sesion y luego revisa el rol
app.use('/api/productos', requiereSesion, productoRoutes);
app.use('/api/ventas', requiereSesion, permitirRoles('comandante', 'cajero'), ventaRoutes);
app.use('/api/clientes', requiereSesion, permitirRoles('comandante', 'cajero'), clienteRoutes);
app.use('/api/paypal', requiereSesion, permitirRoles('comandante', 'cajero'), paypalRoutes);
app.use('/api/compras', requiereSesion, permitirRoles('comandante', 'almacenista'), compraRoutes);
app.use('/api/finanzas', requiereSesion, permitirRoles('comandante'), finanzasRoutes);
app.use('/api/empleados', requiereSesion, permitirRoles('comandante', 'tripulacion'), empleadoRoutes);

app.use(rutaNoEncontrada);
app.use(manejadorErrores);

```

BACKEND/SERVER.JS LÍNEAS 47–50 DE 50 · FIN

```
47  const puerto = process.env.PUERTO || 3200;
48  app.listen(puerto, () => {
49    console.log('Servidor ejecutandose en http://localhost:' + puerto);
50  });
```

• BACKEND/MIDDLEWARE/AUTH.MIDDLEWARE.JS

BACKEND/MIDDLEWARE/AUTH.MIDDLEWARE.JS LÍNEAS 1–48 DE 48 · FIN

```
1  // Seguridad: revisa la sesion y el rol antes de entrar a cada ruta
2  const jwt = require('jsonwebtoken');
3
4  // Solo deja pasar si la cookie trae un token valido
5  function requiereSesion(peticion, respuesta, siguiente) {
6    const token = peticion.cookies.sesion;
7    if (!token) {
8      respuesta.status(401).json({ mensaje: 'Inicia sesion para continuar' });
9      return;
10   }
11
12   try {
13     peticion.usuario = jwt.verify(token, process.env.JWT_SECRETO);
14     siguiente();
15   } catch (error) {
16     respuesta.status(401).json({ mensaje: 'La sesion vencio, vuelve a entrar' });
17   }
18 }
19
20 // Solo deja pasar a los roles indicados. Se usa despues de requiereSesion
21 function permitirRoles(...roles) {
22   return (peticion, respuesta, siguiente) => {
23     if (!roles.includes(peticion.usuario.rol)) {
24       respuesta.status(403).json({ mensaje: 'Tu rol no tiene acceso a este modulo' });
25       return;
26     }
27     siguiente();
28   };
29 }
30
31 module.exports = { requiereSesion, permitirRoles };
32
33 // Maximo 5 intentos de login por minuto desde la misma IP
34 const intentos = {};
35 function limitarIntentos(peticion, respuesta, siguiente) {
36   const ahora = Date.now();
37   const ip = peticion.ip;
38   intentos[ip] = (intentos[ip] || []).filter(t => ahora - t < 60000);
39
40   if (intentos[ip].length >= 5) {
41     respuesta.status(429).json({ mensaje: 'Demasiados intentos, espera un minuto' });
42     return;
43   }
44   intentos[ip].push(ahora);
45   siguiente();
46 }
47
48 module.exports.limitarIntentos = limitarIntentos;
```

• BACKEND/MIDDLEWARE/ERRORES.MIDDLEWARE.JS

BACKEND/MIDDLEWARE/ERRORES.MIDDLEWARE.JS LÍNEAS 1-30 DE 30 · FIN

```
1  // Errores: ninguna peticion se queda colgada y el servidor no se cae
2
3  // Ruta que no existe
4  function rutaNoEncontrada(peticion, respuesta) {
5    respuesta.status(404).json({ mensaje: 'Ruta no encontrada: ' + peticion.originalUrl });
6  }
7
8  // Cualquier error que llegue con next(error)
9  function manejadorErrores(error, peticion, respuesta, siguiente) {
10   console.error(error);
11
12   // JSON mal escrito en el cuerpo
13   if (error.type === 'entity.parse.failed') {
14     respuesta.status(400).json({ mensaje: 'El cuerpo de la peticion no es JSON valido' });
15     return;
16   }
17   // Errores tipicos de MySQL
18   if (error.code === 'ER_DUP_ENTRY') {
19     respuesta.status(409).json({ mensaje: 'Ese registro ya existe' });
20     return;
21   }
22   if (error.code === 'ER_ROW_IS_REFERENCED_2') {
23     respuesta.status(409).json({ mensaje: 'No se puede borrar, hay registros que dependen de
   este' });
24     return;
25   }
26
27   respuesta.status(error.estado || 500).json({ mensaje: error.message || 'Error en el servidor'
   });
28 }
29
30 module.exports = { rutaNoEncontrada, manejadorErrores };
```

• FRONTEND/SRC/APP/SERVICIOS/GUARDS.TS

FRONTEND/SRC/APP/SERVICIOS/GUARDS.TS LÍNEAS 1–25 DE 25 · FIN

```ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { SesionService } from './sesion.service';

// Sin sesion no se entra a ninguna pantalla del ERP
export const sesionGuard: CanActivateFn = async () => {
  const sesion = inject(SesionService);
  const router = inject(Router);
  return (await sesion.cargar()) ? true : router.parseUrl('/login');
};

// Cada rol solo entra a los modulos que tiene en modulos.ts
export const rolGuard: CanActivateFn = async (ruta) => {
  const sesion = inject(SesionService);
  const router = inject(Router);
  await sesion.cargar();
  return sesion.puedeAbrir('/' + ruta.routeConfig?.path) ? true : router.parseUrl('/inicio');
};

// Si ya hay sesion, el login manda directo al inicio
export const invitadoGuard: CanActivateFn = async () => {
  const sesion = inject(SesionService);
  const router = inject(Router);
  return (await sesion.cargar()) ? router.parseUrl('/inicio') : true;
};
```

08 Evidencia de funcionamiento

Capturas tomadas de la aplicación corriendo en mi laptop: backend con `node server.js` (puerto 3200), frontend con `npx ng serve` (puerto 4200) y la base `petrova_erp` en MariaDB de XAMPP. Las salidas de terminal son reales.

• 8.1 INSTALACIÓN Y ARRANQUE

NODE, NPM Y DEPENDENCIAS DEL BACKEND — SALIDA REAL

```
PS> node -v
v24.19.0
PS> npm -v
11.17.0

PS backend> npm ls --depth=0
back-erp@1.0.0 C:\Users\mi compu\Downloads\Autenticacion\erp-cafe-petrova\backend
+-- bcryptjs@3.0.3
+-- cookie-parser@1.4.7
+-- cors@2.8.6
+-- dotenv@18.0.9
+-- express@4.22.3
+-- jsonwebtoken@9.0.3
+-- mysql2@3.24.6
`-- nodemailer@10.2.1
```

IMPORTACIÓN DE LA BASE — SALIDA REAL

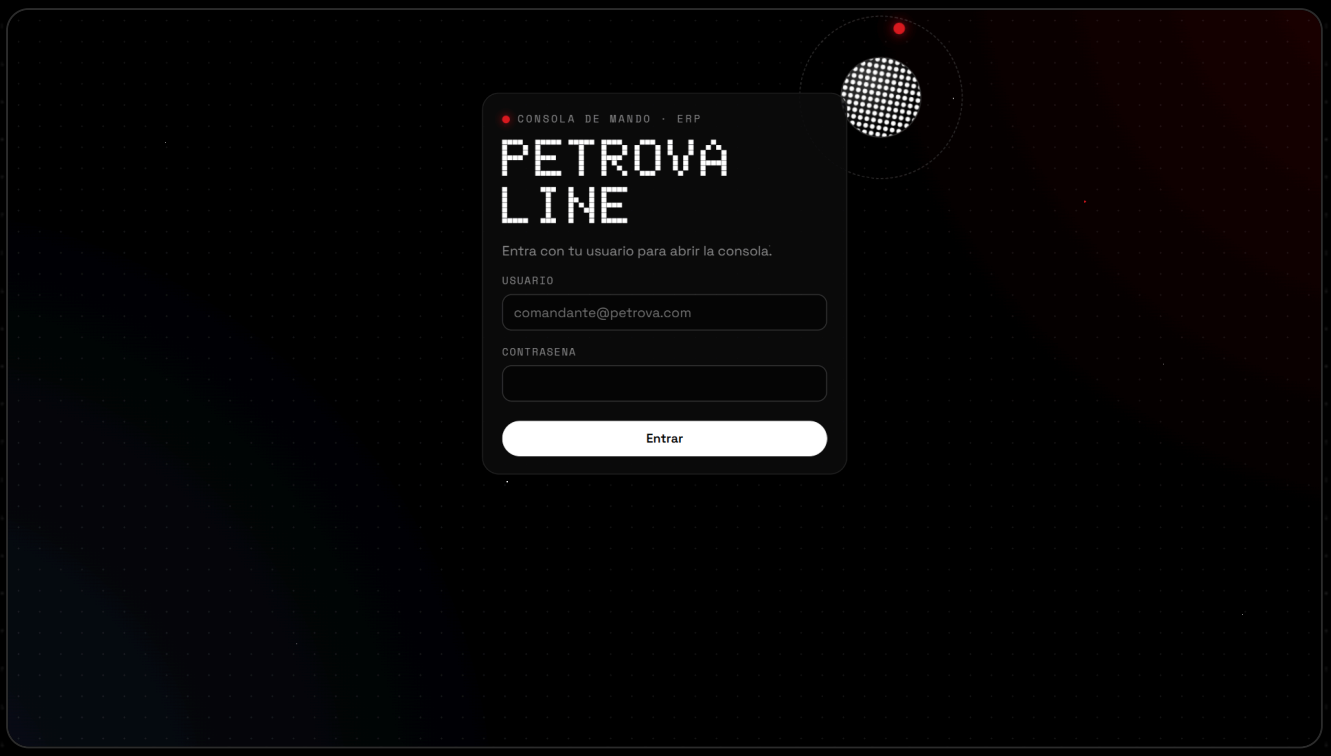
```
PS> mysql -u root < backend/sql/petrova_erp.sql
(sin errores)

PS> mysql -u root petrova_erp -e "SHOW TABLES; SELECT correo, LEFT(contrasena,29) AS hash_bcrypt, rol
FROM usuario;"
+-----------------------+
| Tables_in_petrova_erp |
+-----------------------+
| cliente               |
| compra                |
| detalle_compra        |
| detalle_venta         |
| empleado              |
| merma                 |
| movimiento            |
| producto              |
| proveedor             |
| rotacion              |
| usuario               |
| venta                 |
+-----------------------+
+-------------------------+---------------------------------+-------------+
| correo                  | hash_bcrypt                     | rol         |
+-------------------------+---------------------------------+-------------+
| comandante@petrova.com  | $2b$10$R54SbcaPQzJhooC5t/vxe.... | comandante  |
| cajero@petrova.com      | $2b$10$R54SbcaPQzJhooC5t/vxe.... | cajero      |
| almacen@petrova.com     | $2b$10$R54SbcaPQzJhooC5t/vxe.... | almacenista |
| tripulacion@petrova.com | $2b$10$R54SbcaPQzJhooC5t/vxe.... | tripulacion |
+-------------------------+---------------------------------+-------------+
```

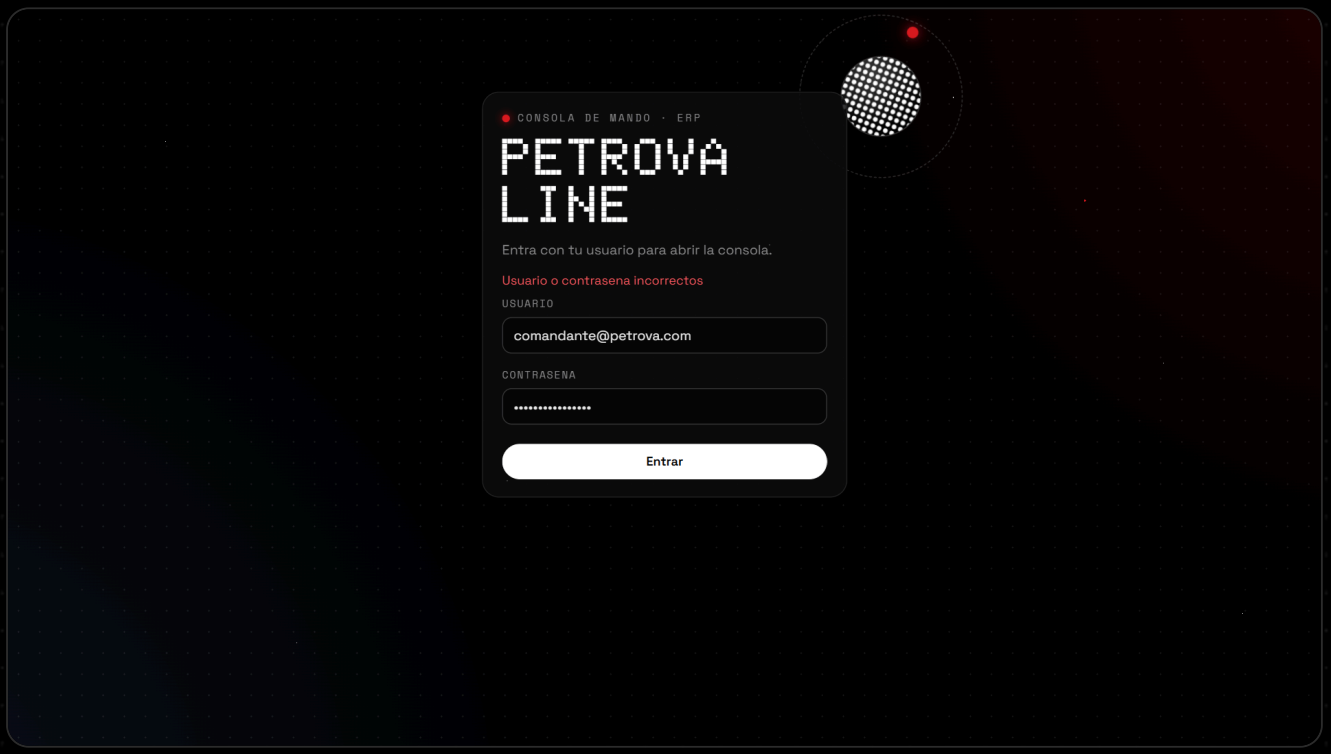
SERVIDOR INICIADO SALIDA REAL

```
PS backend> node server.js
◇ injected env (15) from .env
Servidor ejecutandose en http://localhost:3200
Conectado a MySQL
```

• 8.2 INICIO DE SESIÓN

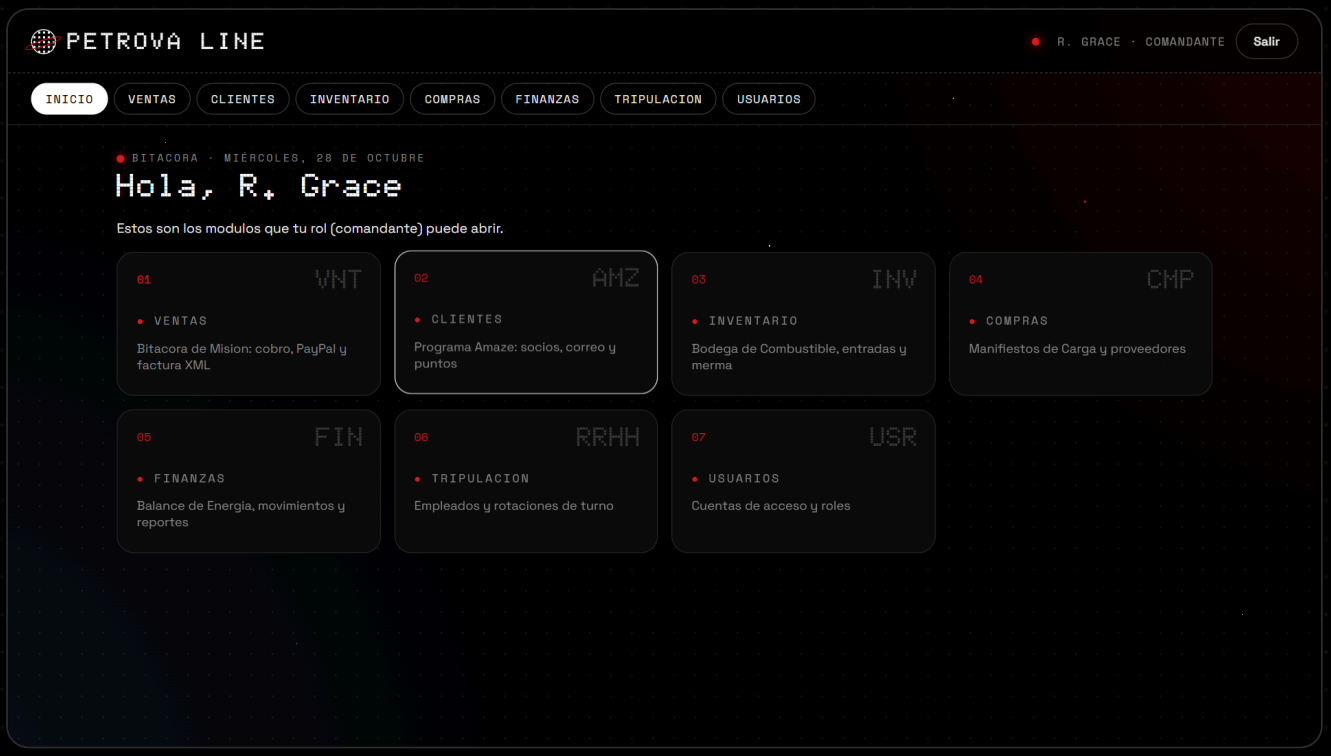


Evidencia 1. Pantalla de login de la consola de mando



Evidencia 2. Contraseña incorrecta: el backend responde 401 y se muestra el aviso

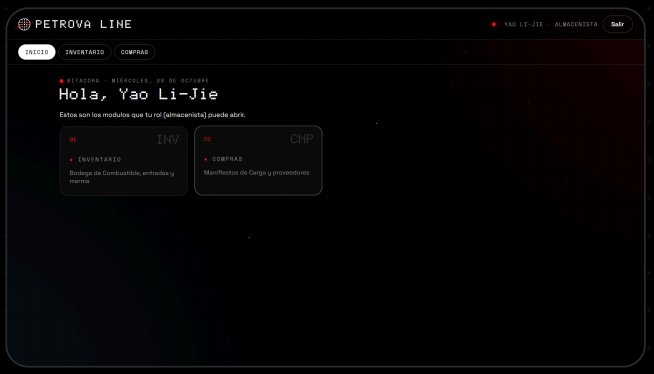
• 8.3 MENÚ SEGÚN EL ROL



Evidencia 3. Comandante: ve los 7 módulos



Evidencia 4. Cajero: solo Ventas y Clientes



Evidencia 5. Almacenista: Inventario y Compras

Al escribir `/finanzas` en la URL con la sesión del cajero, el `rolGuard` lo regresa a `http://localhost:4200/inicio`, y si intenta pedir los datos directo a la API la respuesta es: `403 {"mensaje":"Tu rol no tiene acceso a este modulo"}`.

• 8.4 PRUEBAS DE SEGURIDAD DE LA API (MIDDLEWARE)

CURL CONTRA LA API · REQUIERESESION, PERMITIRROLES, PRECIOS Y ERRORES — SALIDA REAL

```
$ curl http://localhost:3200/api/productos            # sin cookie de sesion
{"mensaje":"Inicia sesion para continuar"}   -> HTTP 401

$ curl -X POST /api/usuarios/login  {"correo":"cajero@petrova.com","contrasena":"petrova123"}
{"id":2,"idEmpleado":2,"correo":"cajero@petrova.com","rol":"cajero","nombre":"Ilyukhina Petrova"}   ->
HTTP 200
$ cat cookies.txt   # el token va en una cookie httpOnly
#HttpOnly_localhost  sesion=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.e...

$ curl -b cookies.txt /api/finanzas/balance          # cajero entra a Finanzas
{"mensaje":"Tu rol no tiene acceso a este modulo"}   -> HTTP 403

$ curl -b cookies.txt /api/usuarios                  # cajero entra a Usuarios
{"mensaje":"Tu rol no tiene acceso a este modulo"}   -> HTTP 403

$ curl -b cookies.txt -X POST /api/ventas  {"productos":[{"idProducto":1,"cantidad":2,"precio":0.01}],
...}
# el navegador manda precio 0.01; el servidor usa el de la base (62.00)
{"id":8,"subtotal":124,"iva":19.84,"total":143.84}   -> HTTP 200

$ curl -b cookies.txt -X POST /api/ventas  {"productos":[{"idProducto":7,"cantidad":99}]}
{"mensaje":"No hay suficiente Chocolate Eridiano (quedan 4)"}   -> HTTP 400

$ curl -b cookies.txt -X POST /api/ventas  {malo
{"mensaje":"El cuerpo de la peticion no es JSON valido"}   -> HTTP 400

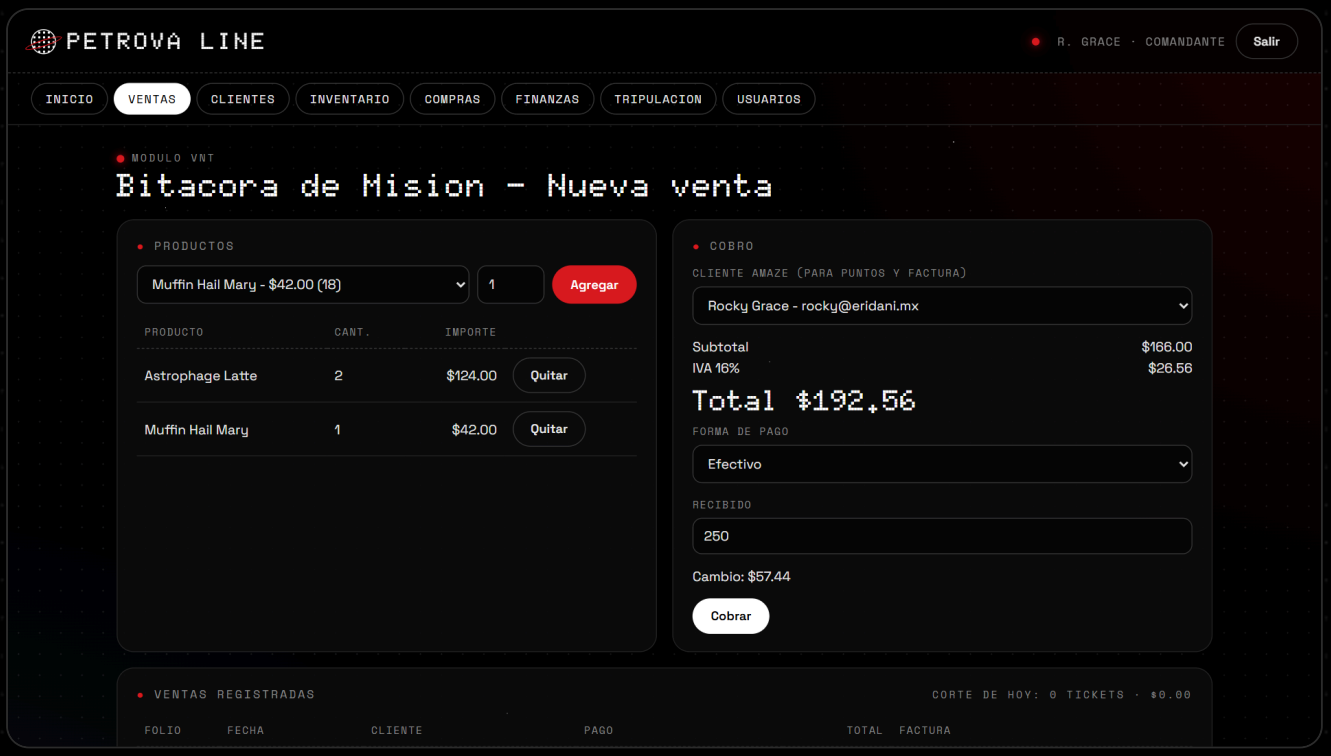
$ curl -b cookies.txt /api/no-existe
{"mensaje":"Ruta no encontrada: /api/no-existe"}   -> HTTP 404

$ curl -b "sesion=token-inventado" /api/ventas
{"mensaje":"La sesion vencio, vuelve a entrar"}   -> HTTP 401
```

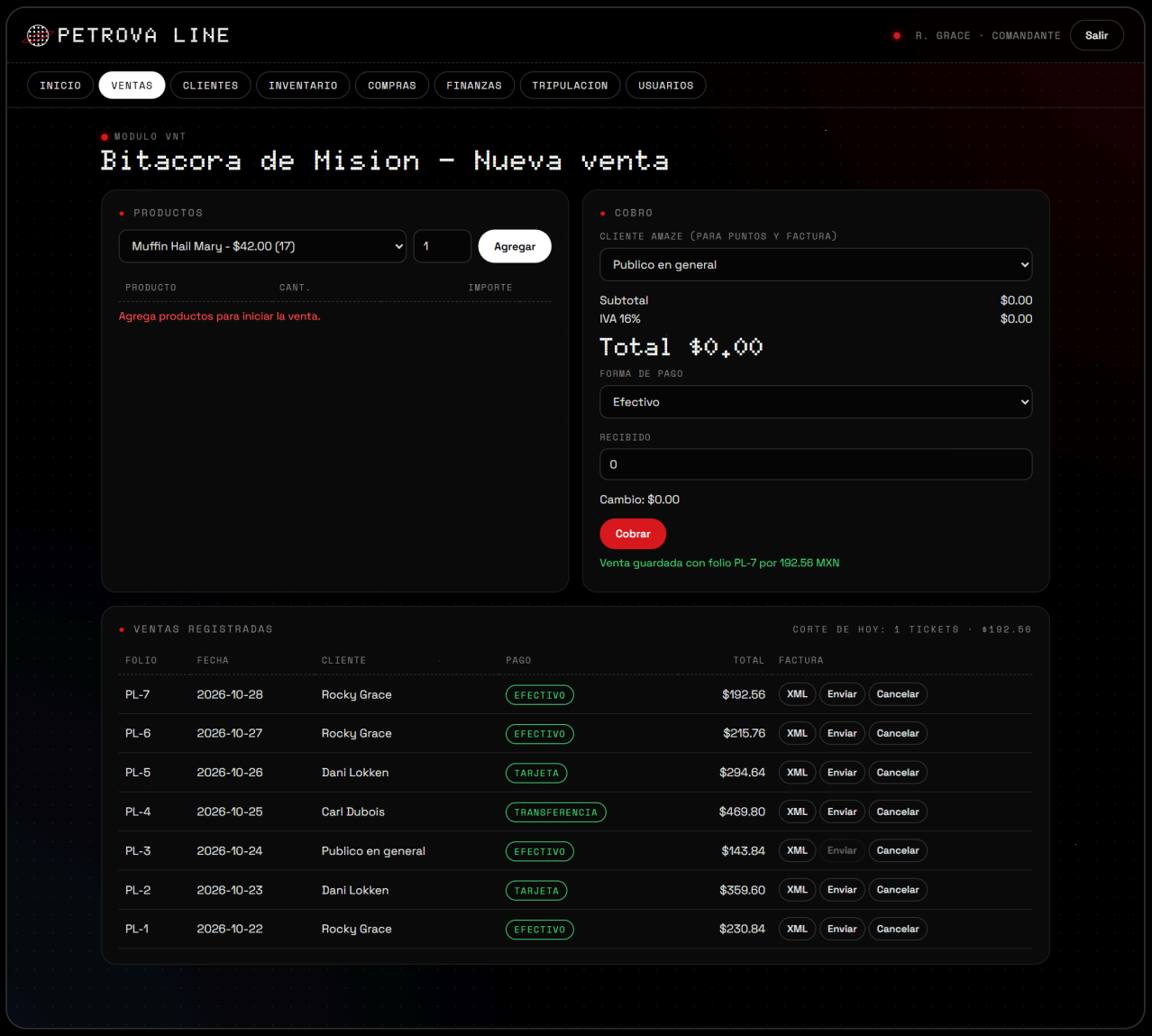
LIMITARINTENTOS · FUERZA BRUTA EN EL LOGIN — SALIDA REAL

```
$ for i in 1..6; do curl -X POST /api/usuarios/login
{"correo":"comandante@petrova.com","contrasena":"adivina"}; done
{"mensaje":"Usuario o contrasena incorrectos"}   -> HTTP 401
{"mensaje":"Usuario o contrasena incorrectos"}   -> HTTP 401
{"mensaje":"Usuario o contrasena incorrectos"}   -> HTTP 401
{"mensaje":"Usuario o contrasena incorrectos"}   -> HTTP 401
{"mensaje":"Demasiados intentos, espera un minuto"}   -> HTTP 429
{"mensaje":"Demasiados intentos, espera un minuto"}   -> HTTP 429
```

• 8.5 VENTAS

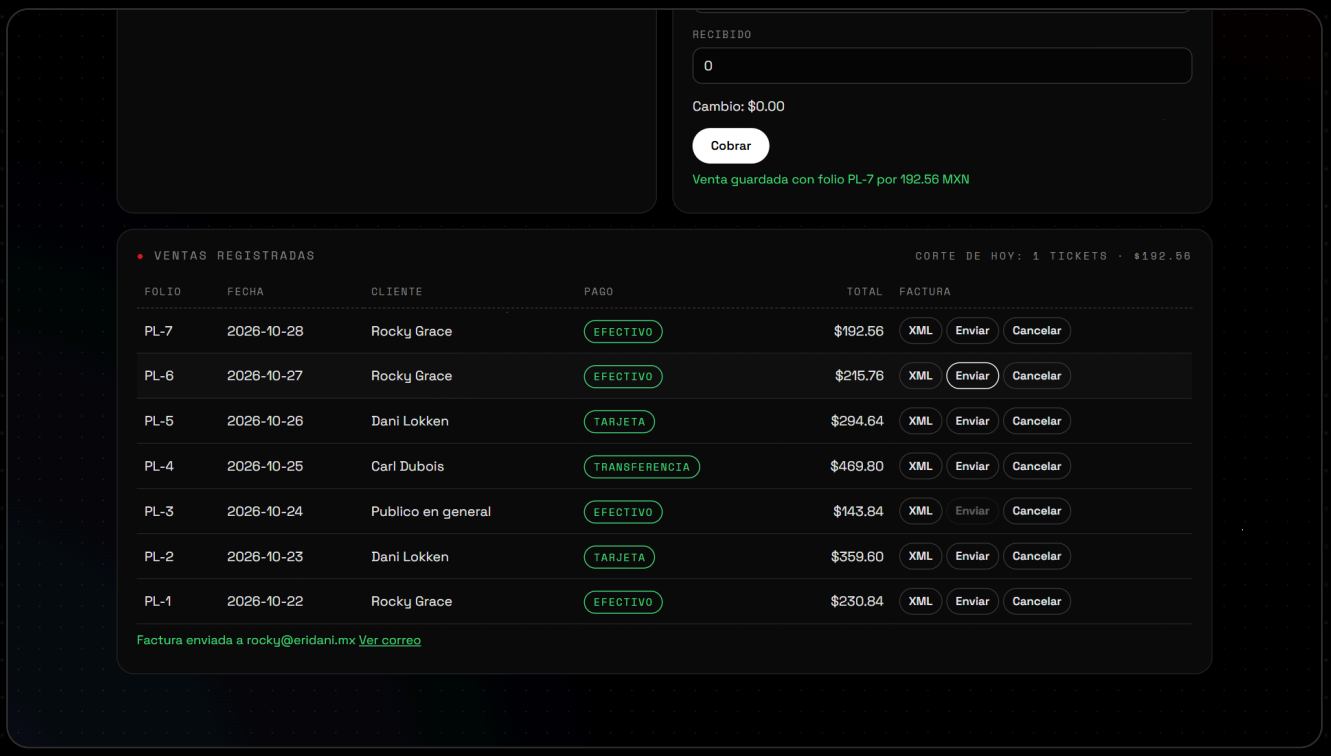


Evidencia 6. Ticket con dos productos, cliente Amaze, IVA y cambio

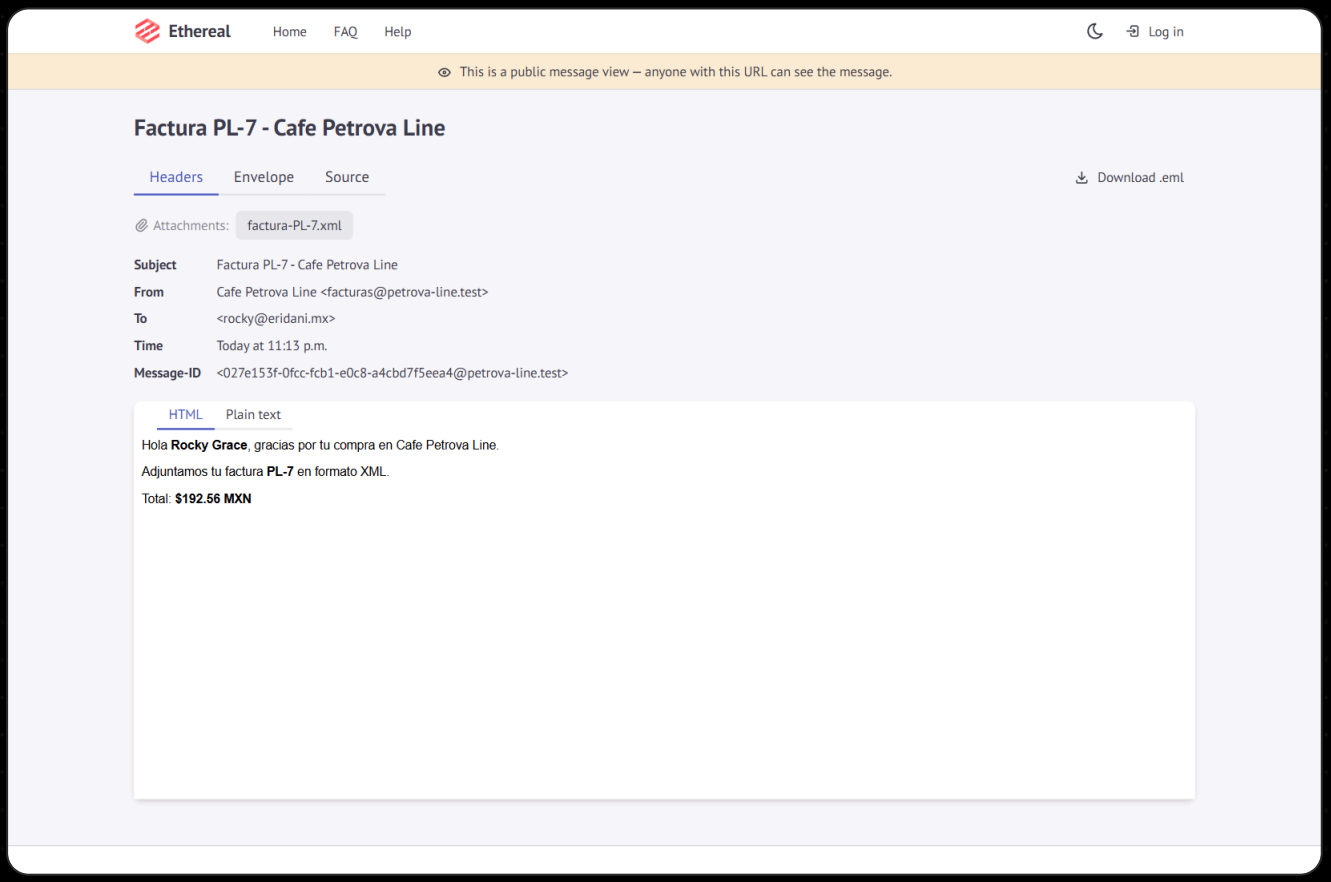


Evidencia 7. Venta guardada con folio PL-7; aparece en la lista y en el corte del día

• 8.6 FACTURA XML POR CORREO

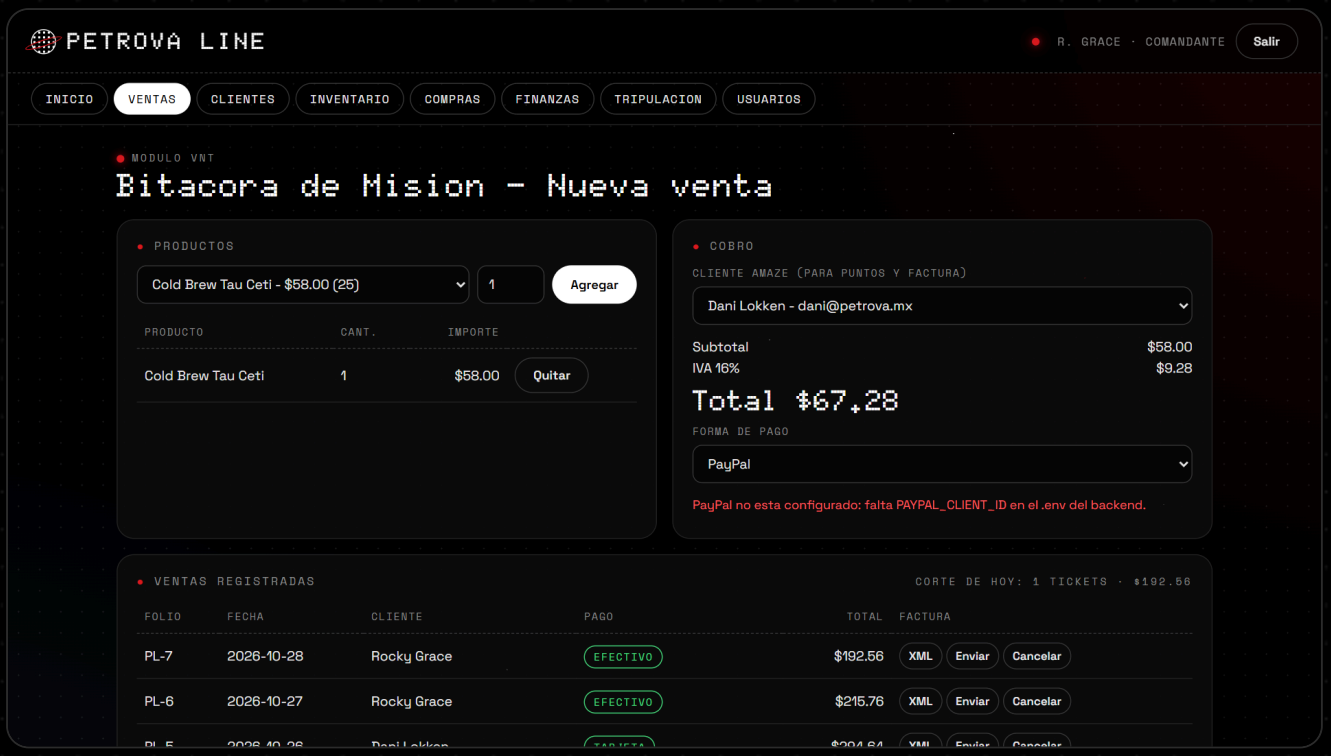


Evidencia 8. Botón Enviar: la API confirma el envío y da la liga del correo



Evidencia 9. Correo recibido con factura-PL-7.xml adjunta (bandeja de prueba Ethereal)

8.7 PAYPAL



Evidencia 10. Forma de pago PayPal en el punto de venta

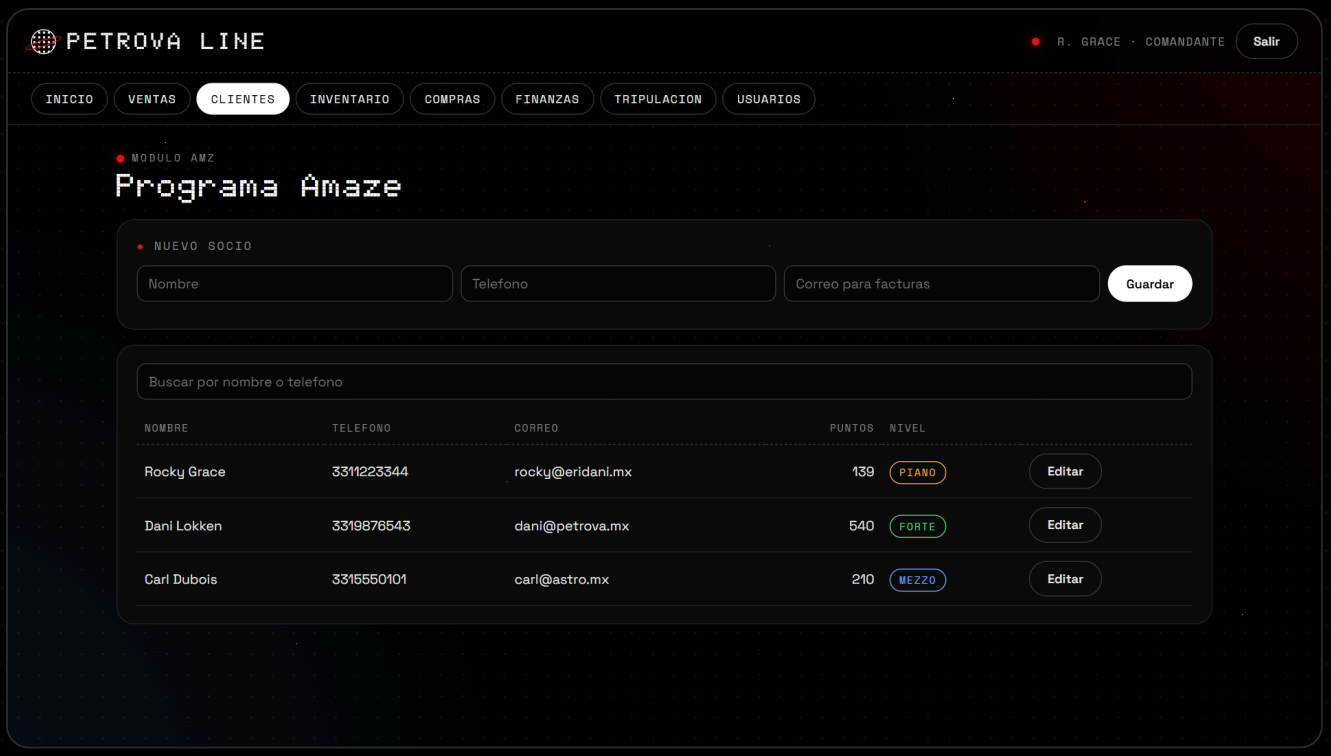
RUTAS DE PAYPAL — SALIDA REAL

```
$ curl -b cookies.txt -X POST /api/paypal/orden  {"productos":[{"idProducto":6,"cantidad":1}]}
{"mensaje":"PayPal no esta configurado en el .env"}   -> HTTP 503

$ curl -b cookies.txt /api/paypal/config
{"clientId":"","moneda":"MXN"}   -> HTTP 200
```

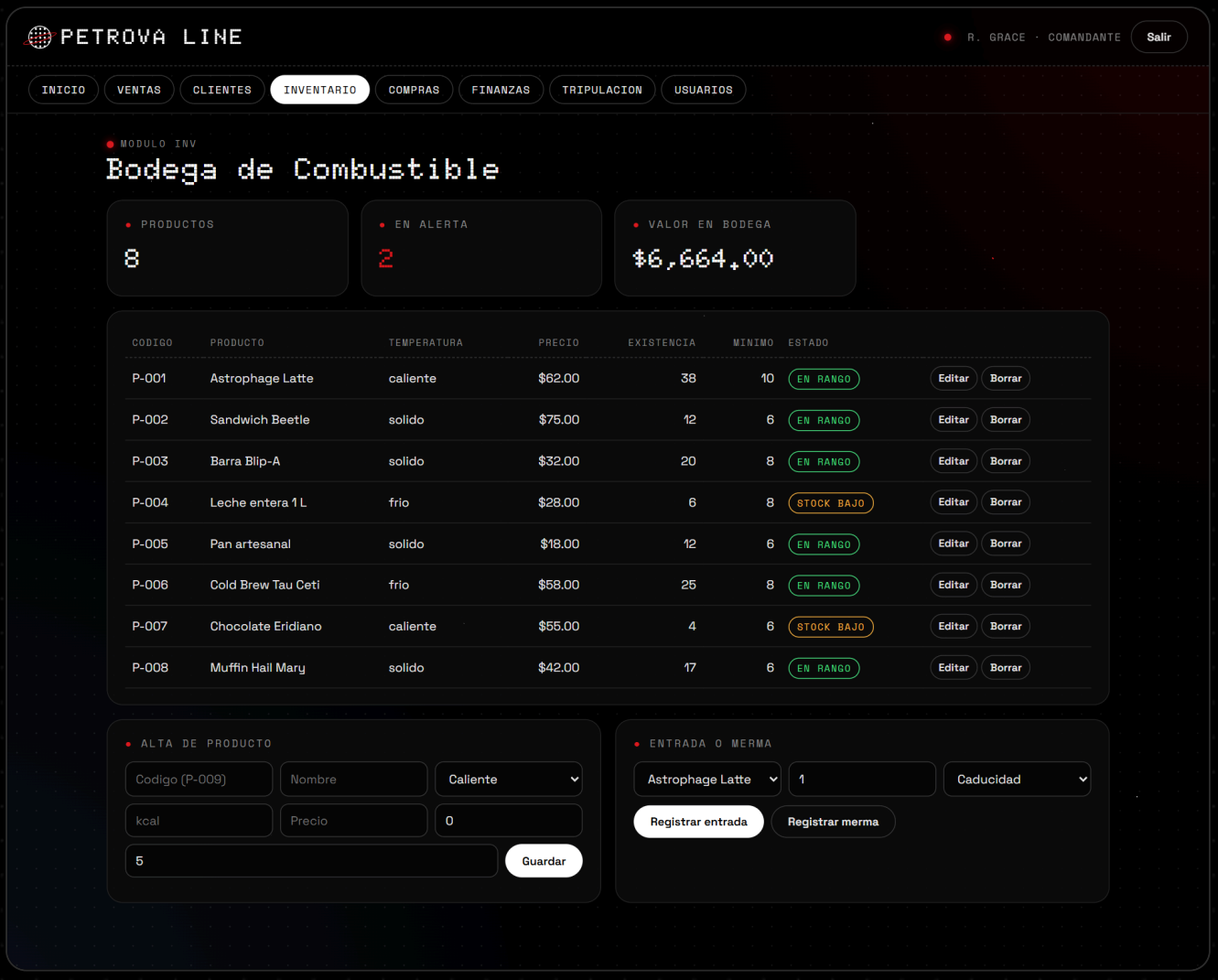
> Para la demostración en clase se colocan las credenciales sandbox en `backend/.env` (`PAYPAL_CLIENT_ID` y `PAYPAL_CLIENT_SECRET`); con ellas el SDK pinta los botones de PayPal y la venta queda con `forma_pago = paypal` y el id de captura en la factura (`idPagoPaypal`).

• 8.8 CLIENTES (PROGRAMA AMAZE)



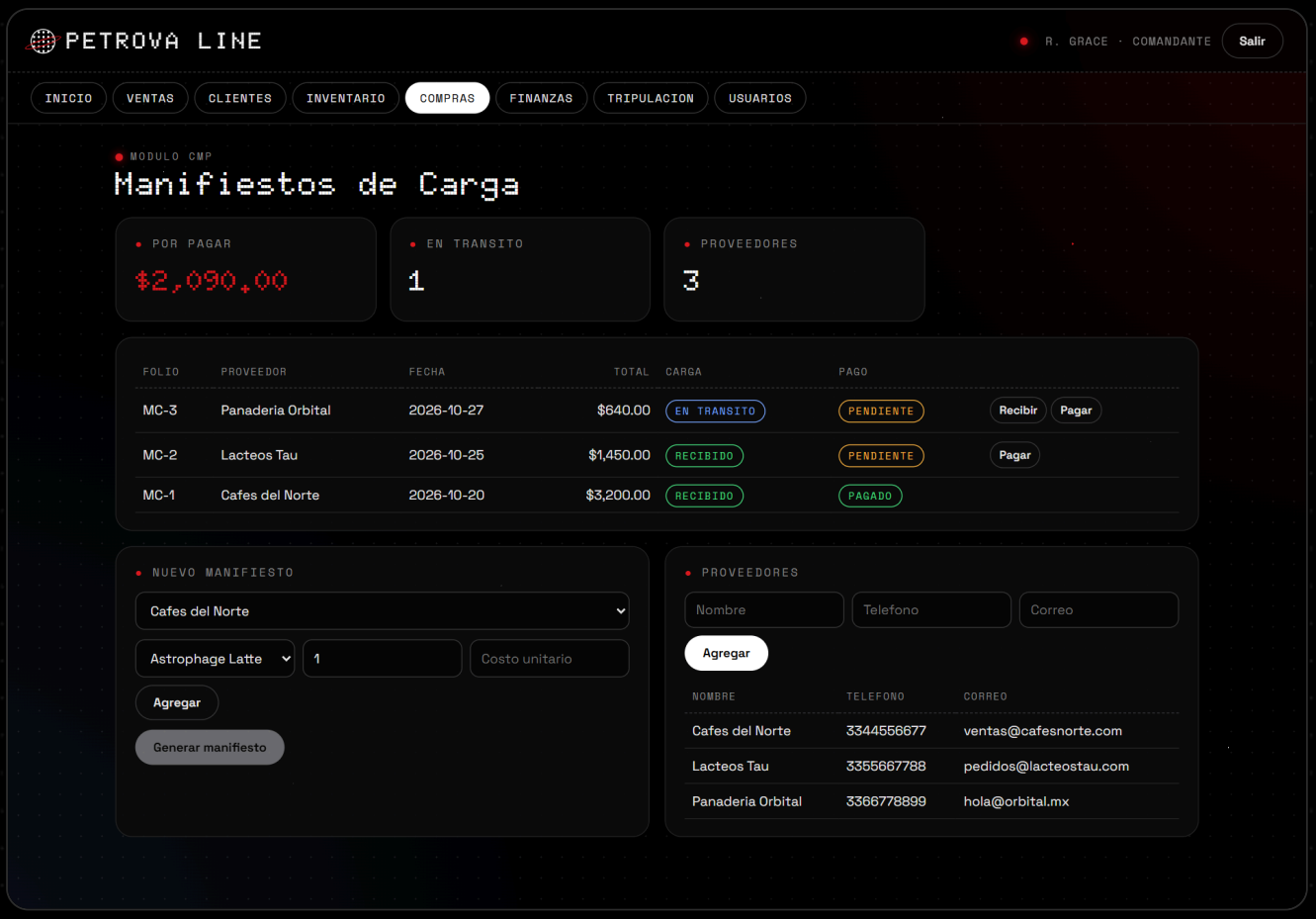
Evidencia 11. Socios con correo para facturas, puntos y nivel

• 8.9 INVENTARIO



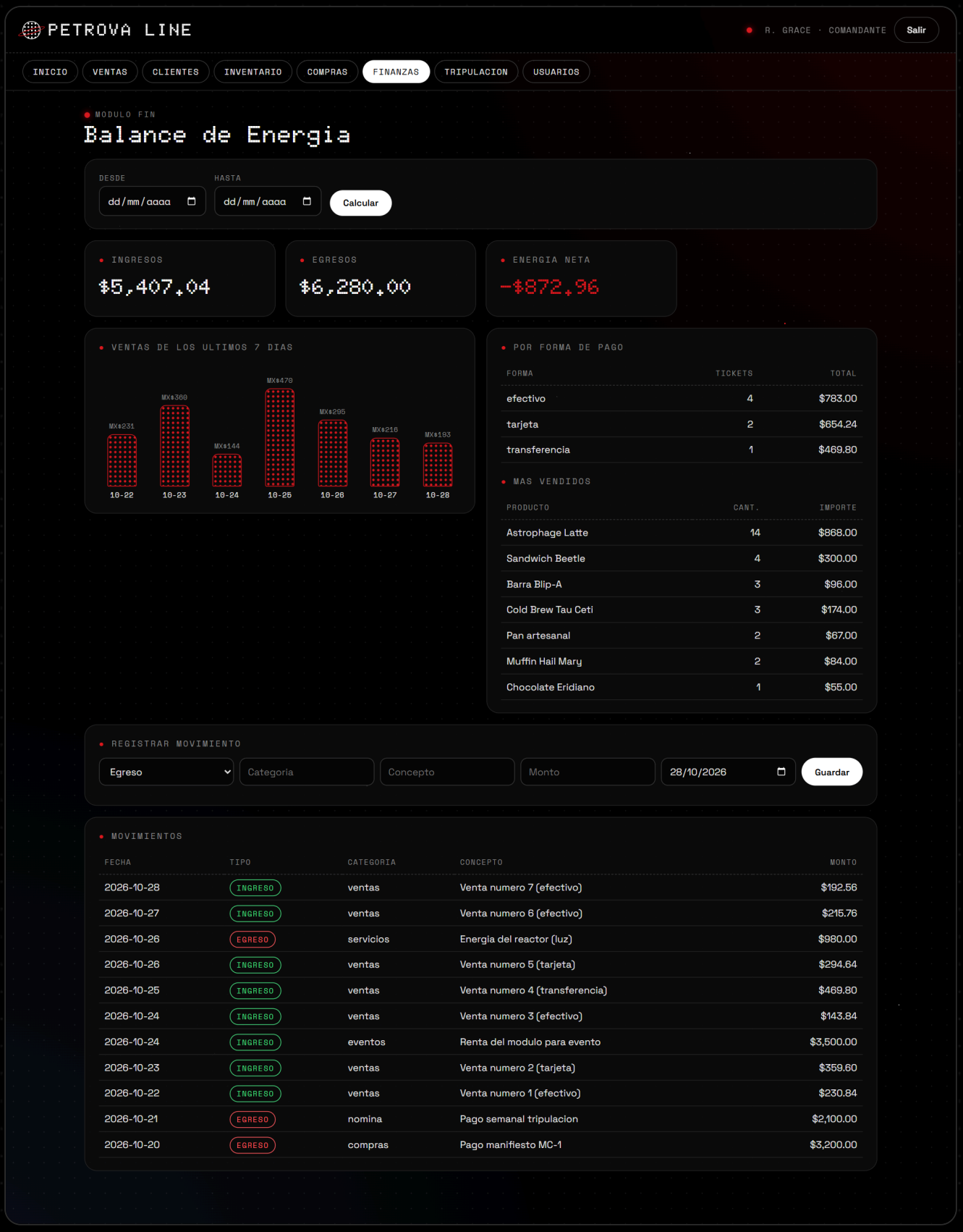
Evidencia 12. Bodega de Combustible con alertas de stock mínimo, alta, entrada y merma

• 8.10 COMPRAS Y PROVEEDORES



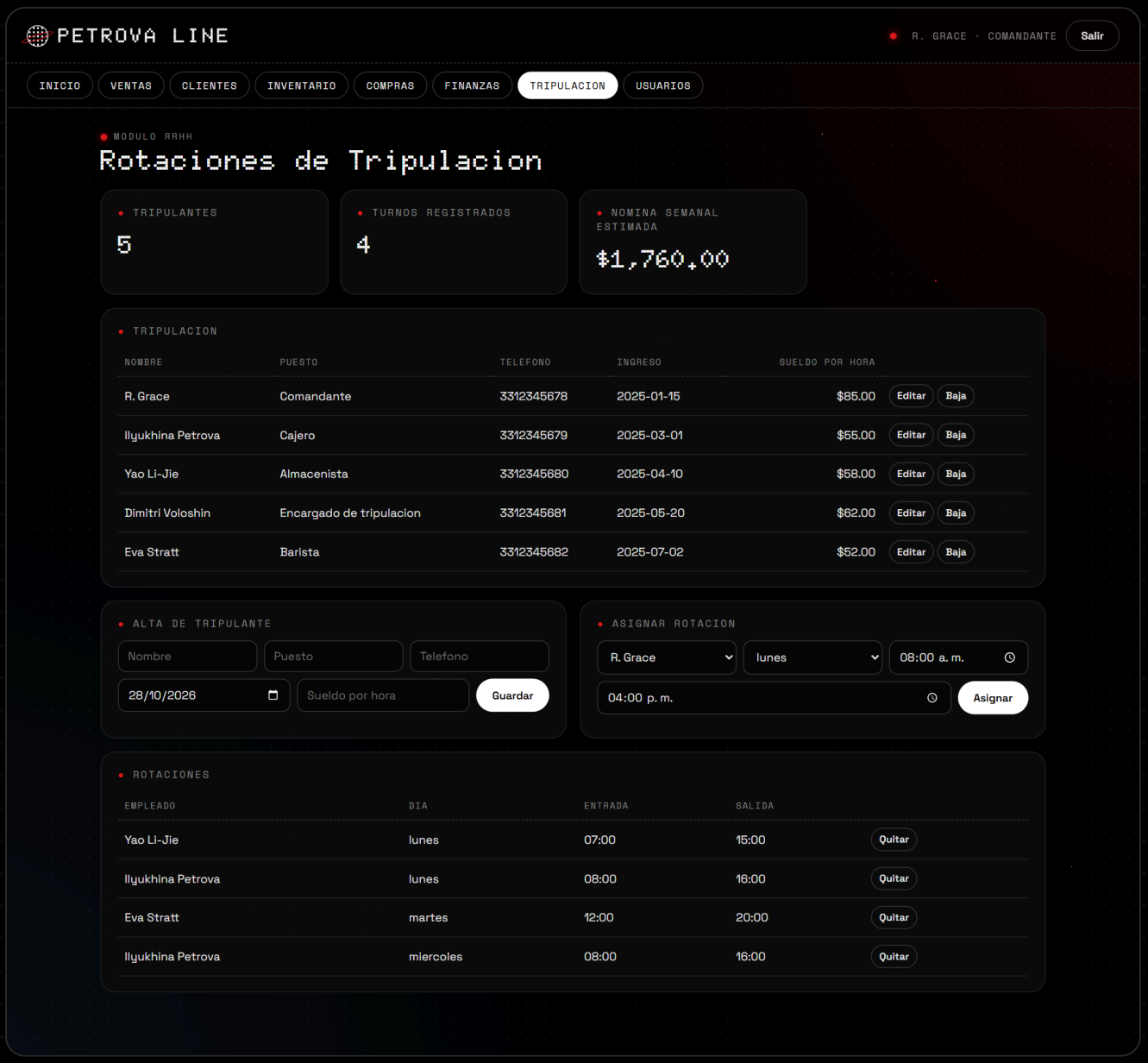
Evidencia 13. Manifiestos de carga, estado de recepción y pago, y proveedores

• 8.11 FINANZAS Y CONTABILIDAD



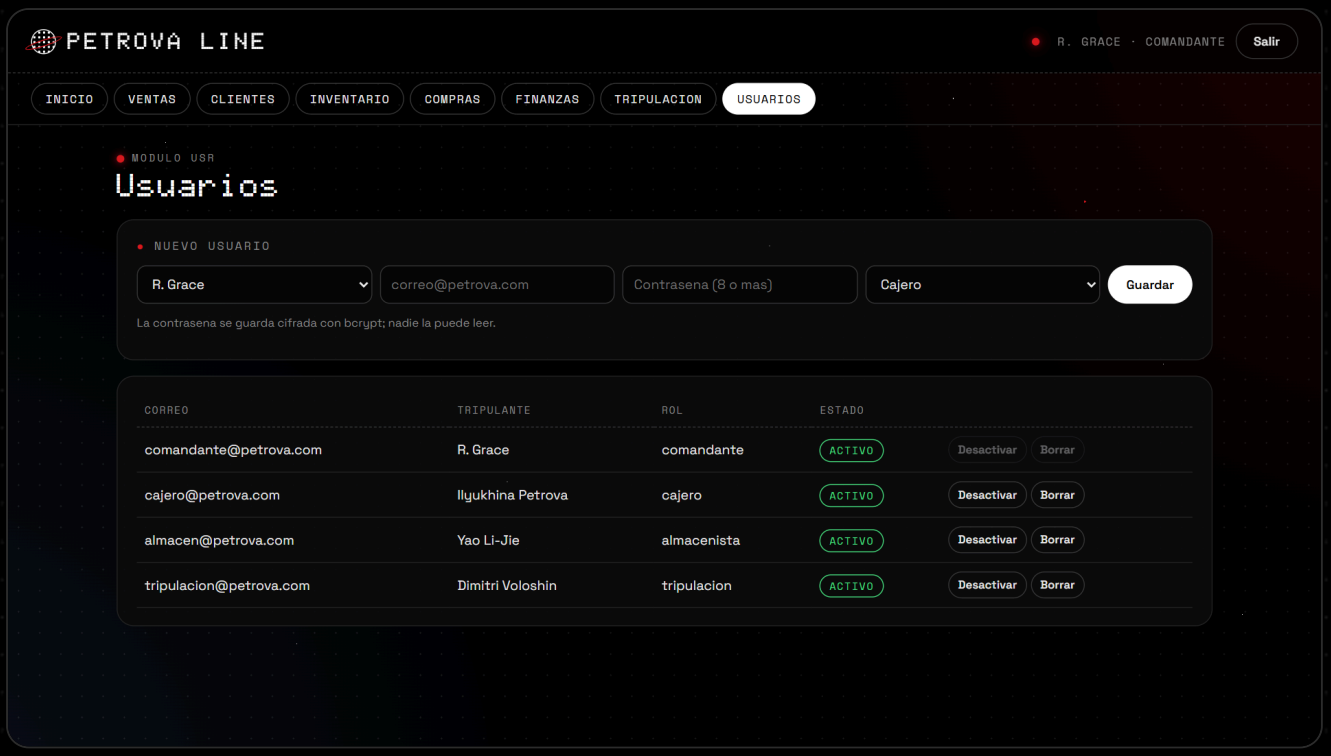
Evidencia 14. Balance de Energia, ventas de 7 dias, formas de pago, más vendidos y movimientos

• 8.12 RRHH (TRIPULACIÓN)



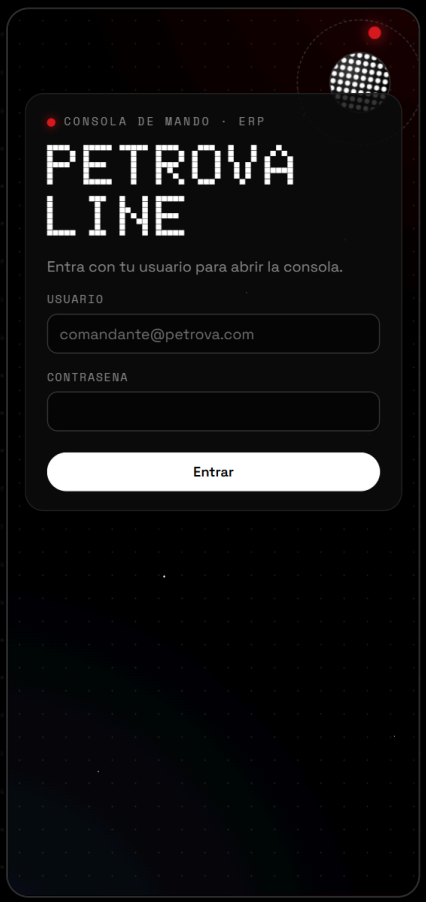
Evidencia 15. Tripulantes, alta, rotaciones y nómina semanal estimada

8.13 USUARIOS



Evidencia 16. Cuentas, roles y estado; la propia cuenta no se puede borrar

8.14 CELULAR



Evidencia 17. Login a 390 px



Evidencia 18. Inicio del cajero



Evidencia 19. Punto de venta

09 Repositorio y ejecución

Repositorio: **github.com/YaelStone/erp-cafe-petrova**

ESTRUCTURA DEL REPOSITORIO — SALIDA REAL

```
PS> tree /F erp-cafe-petrova   (sin node_modules)
erp-cafe-petrova
├── backend/
│   ├── config/
│   │   └── db.js
│   ├── middleware/
│   │   ├── auth.middleware.js
│   │   └── errores.middleware.js
│   ├── models/
│   │   ├── cliente.model.js
│   │   ├── compra.model.js
│   │   ├── empleado.model.js
│   │   ├── movimiento.model.js
│   │   ├── producto.model.js
│   │   ├── usuario.model.js
│   │   └── venta.model.js
│   ├── routes/
│   │   ├── cliente.routes.js
│   │   ├── compra.routes.js
│   │   ├── empleado.routes.js
│   │   ├── finanzas.routes.js
│   │   ├── paypal.routes.js
│   │   ├── producto.routes.js
│   │   ├── usuario.routes.js
│   │   └── venta.routes.js
│   ├── services/
│   │   ├── cliente.service.js
│   │   ├── compra.service.js
│   │   ├── empleado.service.js
│   │   ├── factura.service.js
│   │   ├── finanzas.service.js
│   │   ├── paypal.service.js
│   │   ├── producto.service.js
│   │   ├── usuario.service.js
│   │   └── venta.service.js
│   ├── sql/
│   │   └── petrova_erp.sql
│   ├── .env.example
│   ├── .gitignore
│   ├── README.md
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── paginas/
│   │   │   │   ├── clientes.component.ts
│   │   │   │   ├── compras.component.ts
│   │   │   │   ├── finanzas.component.ts
│   │   │   │   ├── inicio.component.ts
│   │   │   │   ├── inventario.component.ts
│   │   │   │   ├── login.component.ts
│   │   │   │   ├── tripulacion.component.ts
│   │   │   │   ├── usuarios.component.ts
│   │   │   │   └── ventas.component.ts
│   │   │   ├── servicios/
```

```
|   |   |   |   ├── api.service.ts
|   |   |   |   ├── guards.ts
|   |   |   |   ├── sesion.interceptor.ts
|   |   |   |   └── sesion.service.ts
|   |   |   ├── app.component.ts
|   |   |   ├── app.routes.ts
|   |   |   ├── modelos.ts
|   |   |   └── modulos.ts
|   |   ├── index.html
|   |   ├── main.ts
|   |   └── styles.css
|   ├── .gitignore
|   ├── angular.json
|   ├── package.json
|   ├── proxy.conf.json
|   ├── tsconfig.app.json
|   └── tsconfig.json
└── README.md
```

• CÓMO CORRERLO

```
1. phpMyAdmin → Importar → backend/sql/petrova_erp.sql      (crea la base petrova_erp)
2. cd backend
   npm install
   copy .env.example .env      → llenar PAYPAL_* y SMTP_*
   node server.js              → http://localhost:3200
3. cd frontend
   npm install
   npx ng serve                → http://localhost:4200

Usuarios de prueba (contraseña petrova123):
   comandante@petrova.com · cajero@petrova.com · almacen@petrova.com · tripulacion@petrova.com
```

10 Matriz de trazabilidad

REQUISITO	PANTALLA / ARCHIVO	EVIDENCIA
RF-01, RF-02	login.component.ts · usuario.routes.js	Ev. 1, 2 · 8.4
RF-03	modulos.ts · guards.ts	Ev. 3, 4, 5
RF-04	usuarios.component.ts	Ev. 16
RF-05 a RF-08, RF-10, RF-11	ventas.component.ts · venta.service.js	Ev. 6, 7
RF-09	ventas.component.ts · paypal.service.js	Ev. 10 · 8.7
RF-12, RF-13	factura.service.js	Ev. 8, 9 · 4.2
RF-14	venta.routes.js	Ev. 7 (botón Cancelar)
RF-15	clientes.component.ts	Ev. 11
RF-16 a RF-19	inventario.component.ts · producto.service.js	Ev. 12
RF-20 a RF-24	compras.component.ts · compra.service.js	Ev. 13
RF-25 a RF-28	finanzas.component.ts · finanzas.service.js	Ev. 14
RF-29, RF-30	tripulacion.component.ts · empleado.service.js	Ev. 15
RNF-01 a RNF-12	middleware/ · server.js · sesion.interceptor.ts	8.4 · sección 07